

3/2错题

这一知识点主要考察 `pr` コマンド（打印格式化命令）的使用方法。与 `fmt` 不同，`pr` 专门用于将文件分割成带有页眉和页码的 **ページ（页面）** 格式，并可以指定显示特定的页面范围。

`pr [オプション] [ファイル名]`

オプション	説明
<code>-l 行数</code>	ヘッダ、フッタを含めたページの行数を指定(デフォルトは66行)
<code>-h 文字列</code>	ヘッダに表示されるファイル名を指定した文字列に変更
<code>+開始ページ [:終了ページ]</code>	開始ページ、終了ページを指定

💡 知识点总结

在 `pr` 命令中，使用 `-l`（Line）来指定一页的行数，而指定页面范围的格式为 `+开始页:结束页`。

🔍 选项解析与详细说明

- `pr -l 30 +1:2 httpd.conf`：正确答案。
 - `-l 30`：设置一页的长度（Length）为 30 行。
 - `+1:2`：指定从第 1 页开始到第 2 页结束的 **ページ範囲（页面范围）**。注意这里的特殊符号是 `+` 开头且中间用 `:` 分隔。
- `pr -l 30 1:2 ...`：错误。页面范围必须以 `+` 号开头。
- `pr -l 30 +1,2 ...`：错误。`pr` 命令不识别逗号 `,` 作为页面范围的分隔符。
- `pr -l 30 +1;2 ...`：错误。分号 `;` 在 Linux 命令行中代表命令的结束，会导致语法错误。

📊 `pr` 命令常用选项背诵表

为了帮你像小学生一样记牢，请记住这些字母的含义：

选项 (日本語)	含义 (中文解释)	记忆口诀
-l [行数]	1ページの 行数 (Length)	Line = Length, 规定页面的高度。
+开始[:结束]	ページ範囲 を指定	+ 号像十字架, 定位到你要的那一页。
-h [标题]	ヘッダ (Header) を指定	Header, 在每一页顶头加个标题。
-d	ダブルスペース (Double space)	Double, 让行与行之间变宽 (双倍行距)。

小学生级记忆法：复印机的设置

想象 `pr` 命令是一台 老式复印机：

- `-l 30`：你告诉复印机：“每张纸（page）只能印 **30 行** 字，多了就换下一张。”
- `+1:2`：你告诉复印机：“别全给我印出来！我只要 **第 1 张到第 2 张**。”那个 `+` 号就像是复印机上的“增加”按钮，用来指定页码。

✅ 正确答案

`pr -l 30 +1:2 httpd.conf`

📝 考试小秘籍

这是 LPI 考试中的一个细节坑：

- 很多命令用减号 `-`，但 `pr` 的页面范围偏偏用 **加号 `+`**。
- 很多命令范围用逗号 `,` 或横杠 `-`，但 `pr` 偏偏用 **冒号 `:`**。
- 记住这个特例：**`+开始:结束`**。

这一知识点主要考察如何为文件内容添加 **行番号（行号）** 输出，特别要注意 **空行（空白行）** 是否需要编号的区别。

オプション	説明	オプション	説明
-b a	空行を含めた全ての行に行番号をつける (catコマンドの-nオプションと同じ)	-n	空行を含めた全ての行に行番号をつける (nlコマンドの「-b a」オプションと同じ)
-b t	空行を除いた行に行番号をつける (catコマンドの-bオプションと同じ) デフォルト	-b	空行を除いた行に行番号をつける (nlコマンドの「-b t」オプションと同じ)

💡 知识点总结

在 Linux 中，`cat -b` 和 `nl`（默认情况下）都用于为 空行を除いた行（除去空行的行） 添加行号。

🔍 选项解析与详细说明

根据你提供的图片资料，我们可以对比各个选项的具体行为：

- `cat -b file`：正确答案。
 - 功能：空行を除いた行に行番号をつける。这与 `nl` 命令的默认行为一致。
- `nl file`：正确答案。
 - 功能：`nl` 命令的默认选项就是 `-b t`，即 空行を除いた行に行番号をつける。
- `nl -b a file`：错误。
 - 功能：空行を含めた全ての行に行番号をつける。这不符合题目“除去空行”的要求。
- `cat -n file`：错误。
 - 功能：空行を含めた全ての行に行番号をつける。
- `num file`：错误。
 - 功能：Linux 中没有名为 `num` 的标准命令用于显示行号。

📊 行号显示命令对照表

为了帮你像小学生一样记牢，请看这个“空行对待方式”的对比：

目标效果	cat 命令选项	nl 命令选项	记忆口诀
仅给非空行编号	-b	(默认) 或 -b t	B 为 Body，只给有内容的身体编号。
给所有行编号	-n	-b a	N 为 Number all，A 为 All。

🧠 小学生级记忆法：排队领帽子

想象文件里的每一行都在排队领“行号帽子”：

1. `cat -b` 和 `nl`：它们比较严，看到“空位置”（空行）会说：“没人的位子不发帽子！”
2. `cat -n` 和 `nl -b a`：它们比较慷慨，不管是有人还是空位，统统都发帽子。
3. 题目要求：是不给空位置发帽子，所以选 `cat -b` 和 `nl`。

✅ 正确答案

cat -b file, nl file

📝 考试小秘籍

记住一个公式：`cat -n == nl -b a`（全编），`cat -b == nl -b t`（除空行）。考试时如果看到「**全て選択**」，这两组对应关系是拿分关键。

这一知识点主要考察 **環境変数PATH**（环境变量 PATH）的作用及其在 **コマンド（命令）** 搜索中的机制。

💡 知识点总结

PATH 变量定义了一组目录列表，当用户输入命令时，Shell 会按照这些目录的 **順番（顺序）** 寻找对应的可执行文件，从而实现无需输入完整路径即可执行命令的功能。

🔍 选项解析与详细说明

- **パスを指定しなくてもコマンドを実行できるようにするために使用される：正确答案。**
 - **理由：**如果 PATH 变量设置正确，你只需要输入 `ls` 而不需要输入 `/usr/bin/ls`。PATH 的存在就是为了让用户省去输入长长路径的麻烦。
 - **シェルは環境変数PATHに定義されたパスから、入力されたコマンドの実行ファイルを順に探す：正确答案。**
 - **理由：**当你输入一个命令时，Shell 会从 PATH 变量定义的第一个目录开始查找，找到了就执行；如果第一个没找到，就去第二个目录找，依此类推。
 - **rootユーザのみが編集することができる：错误。**
 - **理由：**普通用户也可以修改自己的 PATH 变量（例如在 `~/.bash_profile` 中修改），以方便运行自己安装的软件。
 - **デフォルトでユーザのホームディレクトリが設定されている：错误。**
 - **理由：**出于安全考虑，默认的 PATH 通常不包含用户的 **ホームディレクトリ（家目录）**，甚至也不包含当前目录 `.`。
 - **環境変数PATHに定義されていないパスのコマンドは絶対に実行できない：错误。**
 - **理由：**即使不在 PATH 里，你依然可以通过输入 **絶対パス（绝对路径）**（如 `/home/user/mycmd`）或 **相対パス（相对路径）**（如 `./mycmd`）来直接运行它。
-

PATH 变量的“查找逻辑”对比表

为了帮你像小学生一样记牢，我们看看 Shell 是怎么工作的：

你的操作	Shell 的反应 (日本語)	结果
ls	PATH 内のディレクトリを 順に 探す	找到 /usr/bin/ls 并运行。
/bin/date	PATH を無視して、直接その場所を見る	直接运行该路径下的命令。
my_script	PATH に無ければ「command not found」	报错（除非你在 PATH 里加了它的路径）。

小学生级记忆法：找玩具的箱子

想象你有很多玩具（命令可执行文件）放在不同的箱子（目录）里：

1. **PATH**：就是一张“寻宝图”，上面写着：先找客厅的箱子，再找卧室的箱子。
2. **输入命令**：你喊一声“小熊（命令名）”，Shell 就会按照寻宝图的顺序去翻箱子。
3. **不用 PATH 怎么办？**：除非你直接告诉 Shell：“小熊就在客厅左边的第二个格子里（绝对路径）”，否则它离了寻宝图就不知道去哪找了。

正确答案

- シェルは環境変数PATHに定義されたパスから、入力されたコマンドの実行ファイルを順に探す
- パスを指定しなくてもコマンドを実行できるようにするために使用される

考试小秘籍

在 LPI 考试中，关于 PATH 的题目经常会考“顺序”问题。记住：PATH 变量中目录是用 **冒号** `:` 分隔的，且顺序 **从左到右** 非常重要。

这一知识点主要考察 Shell 的 **ヒアドキュメント**（Here Document / 这里的文档）功能，这是一种特殊的 **リダイレクト**（重定向）方式。

知识点总结

ヒアドキュメント 允许用户在脚本或命令行中直接输入多行文本，并将其作为命令的 **標準入力**（标准输入），直到遇到指定的 **終了文字**（结束标识符）为止。

选项解析与详细说明

- 終了文字が現れるまでコマンドの標準入力にを送る：正确答案。
 - 功能：使用 格式。Shell 会读取随后的每一行内容，直到单独出现一行“終了文字”时停止，并将这些内容一次性传给前面的命令。
- ファイルの内容をコマンドの標準入力に送る：错误。
 - 功能：这是普通的输入重定向（使用 ），是从外部文件读取。而ヒアドキュメント是直接在脚本代码内写内容。
- コマンドの標準エラー出力の出力先を、標準出力の出力先と同じにする：错误。
 - 功能：这是 標準エラー出力（标准错误输出）的重定向，符号是 。
- コマンドの標準出力の出力先を、標準エラー出力の出力先と同じにする：错误。
 - 功能：符号是 ，用于将正常输出转到错误通道。

ヒアドキュメント的语法结构

为了帮你像小学生一样记牢，我们看看它的长相：

Bash

代码块

- 1 コマンド <<終了文字
- 2 1行目の内容
- 3 2行目の内容
- 4 終了文字

构成部分 (日本語)	作用 (中文解释)
<<	引导符号，代表“后面这些全是输入”。
終了文字	自定义的暗号（常用 EOF），告诉 Shell 到哪里结束。
中间的内容	这些文字会被当作命令的 標準入力。

小学生级记忆法：讲故事的信封

想象你要给一个叫 的小猫写信（输入数据）：

1. 普通的 ：你把写好的信封寄给它。
2. ヒアドキュメント ：你当着它的面开始讲故事。

3. **終了文字**：你跟它约定：“当我喊‘**讲完了**’的时候，你就不用再听了。”于是你一直讲，直到喊出那声“讲完了”。
-

✅ 正确答案

終了文字が現れるまでコマンドの標準入力に入力を送る

📝 考试小秘籍

在 LPI 考试中，看到 `<<` 符号就要立刻联想到「**ヒアドキュメント**」和「**終了文字**」。它是编写自动化安装脚本（自动填写交互信息）时的神器。

这一知识点主要考察 `cd` (**Change Directory**) 命令在切换目录时的快捷路径表示法，特别是 **チルダ** (`~`) 的用法。

💡 知识点总结

在 Linux 中，`~` (**チルダ**) 符号代表当前登录用户的 **ホームディレクトリ**（家目录），而单独输入 `cd` 命令不带任何参数时，默认效果也是跳转回该目录。

🔍 选项解析与详细说明

- `cd`：正确答案。
 - **理由**：当 `cd` 命令后面不接任何路径时，Shell 会自动将其解释为“回到当前用户的ホームディレクトリ”。
- `cd ~`：正确答案。
 - **理由**：`~` 是ホームディレクトリの缩写符号。执行 `cd ~` 会直接跳转到 `/home/test`。
- `cd ~test`：正确答案。
 - **理由**：格式为 `~用户名`。这表示跳转到指定用户的家目录。因为当前登录的就是 test，所以 `~test` 同样指向 `/home/test`。
- `cd ~/test`：错误。
 - **理由**：这表示进入家目录下的 **名为 test 的子目录**（即 `/home/test/test`）。题目要求是移动到家目录本身。
- `cd test`：错误。
 - **理由**：由于当前在 `/etc`，这个命令会尝试进入 `/etc/test`。除非 `/etc` 下刚好有个叫 test 的目录，否则会报错。

📊 家目录切换指令对比表

为了帮你像小学生一样记牢，请记住这几种“回家”的方式：

指令	目标路径 (日本語)	记忆口诀
cd	自分のホームディレクトリ	简单粗暴，直接回家。
cd ~	自分のホームディレクトリ	~ 像家里的波浪线地毯。
cd ~用户名	指定した人のホームディレクトリ	去某某人的家里串门。
cd ..	1つ上のディレクトリ	往后退一步。

🧠 小学生级记忆法：回家的暗号

想象你在迷宫（文件系统）里走累了，现在想回自己的家（`/home/test`）：

- `cd`：你太累了，什么都不想说，直接念个咒语。咒语默认就会把你送回出生点（家）。
- `cd ~`：你在地图上画了个 `~` 符号，这个符号在 Linux 地图里就是“家”的坐标。
- `cd ~test`：你大喊一声：“去 **test** 的家！” 既然你自己就是 `test`，那你自然就回到了自己的家。

✅ 正确答案

cd, cd ~, cd ~test

📝 考试小秘籍

这是 LPI 考试常考的陷阱题：

- 记住 `cd` 和 `cd ~` 是完全等价的。
- 看到 `cd ~用户名` 时要先看一眼当前是谁在登录。如果用户名和当前用户一致，那它也是回家的路。

这一知识点主要考察使用 **sed コマンド**（流编辑器）进行 **行削除（行删除）** 的匹配规则，特别是针对 **行頭（行首）** 的特殊字符匹配。

💡 知识点总结

在 `sed` 命令中，`d` 指令用于删除匹配的行。要匹配 **行頭（行首）**，必须使用元字符 `^`。

选项解析与详细说明

- `sed /^#/d test.txt`：正确答案。
 - `/^#/`：这是匹配模式。`^` 表示行首，`#` 是我们要找的字符。
 - `d`：表示 **delete**（删除）。
 - **结果**：凡是以 `#` 开头的行都会被删除，剩下的内容被显示出来。
- `sed /#/d test.txt`：错误。
 - **理由**：这个命令会删除 **所有包含** `#` 的行，不论 `#` 是在行首、行中还是行尾。
- `sed -e ^#d test.txt`：错误。
 - **理由**：格式错误。匹配模式通常需要包含在斜杠 `/.../` 中。
- `sed -e ^#/d test.txt`：错误。
 - **理由**：语法不完整且缺少必要的斜杠。
- `sed #d test.txt`：错误。
 - **理由**：语法完全不正确。

sed 匹配模式背诵表

为了帮你像小学生一样记牢，请记住这些“位置”符号：

符号	含义 (日本語)	记忆口诀
<code>^</code>	行頭 (开始)	<code>^</code> 像个尖尖的房顶，盖在开头。
<code>\$</code>	行末 (结束)	<code>\$</code> 是钱，辛苦到最后才能领钱。
<code>d</code>	削除 (Delete)	<code>d</code> 代表 d isappear（消失）。

小学生级记忆法：排队检查员

想象 `sed` 是一个站在校门口的 **检查员**：

1. `/^#/d`：检查员规定：“只要 **排头** (`^`) 的人拿着 **小旗子** (`#`)，就把这一整排人都 **请出去** (`d`)。”
2. `/#/d`：检查员规定：“只要这一排里 **任何位置** 有人拿着小旗子，整排都请出去。”
3. **题目要求**：是“行头”，所以必须得带上那个房顶符号 `^`。

✓ 正确答案

`sed /^#/d test.txt`

📝 考试小秘籍

在 LPI 考试中，`sed` 是必考内容。记住最基础的结构：`sed /模式/动作 文件名`。

如果你想直接修改文件内容而不是仅仅显示出来，记得加上 `-i` 选项（in-place）。

💡 知识点总结

在 `sed` 中，基本的替换格式为 `s/检索文字列/置换文字列/`。默认情况下，它只替换每行的 **最初（第一个）** 匹配项；若要替换一行中出现的 **全て（所有）** 匹配项，必须在末尾加上 `g` (**global**) 标志。

🔍 选项解析与详细说明

根据置换逻辑，我们来筛选正确答案：

- `sed s/pingt/hoge/g test.txt`：正确答案。
 - `s`：代表 substitute（替换）。
 - `g`：代表 global（全局）。没有它，一行中如果出现两次 "pingt"，只有第一个会被改掉。
 - `sed -e s/pingt/hoge/g test.txt`：正确答案。
 - `-e`：指定要执行的脚本（命令）。当只有一个替换动作时，`-e` 是可以省略的，所以这个选项和上面那个效果一样。
 - `sed s/pingt/hoge/ test.txt`：错误。
 - **理由**：缺少 `g`。它只会替换每行中 **第一个** 找到的 "pingt"。
 - `sed -e s/pingt/hoge/ test.txt`：错误。
 - **理由**：同样因为缺少 `g`，无法做到“全部”置换。
 - `grep s/pingt/hoge/ test.txt`：错误。
 - **理由**：`grep` 是用来 **检索（搜索）** 包含特定字符串的行的，它不具备修改或替换文字的功能。
-

📊 sed 置换命令结构表

为了帮你像小学生一样记牢，请记住这个“替换公式”：

指令组成	作用 (日本語)	记忆口诀
s	置換 (substitute)	s 代表 switch，换个样子。
/A/B/	A を B に	从 A 变到 B。
g	全て (global)	g 代表 grand，全都要变！

小学生级记忆法：魔法棒

想象 `sed` 是一根魔法棒：

- `s/pingt/hoge/`：你挥动魔法棒说：“把 pingt 变成 hoge！”魔法棒比较懒，它看到每一行里的第一个 pingt 变了色就停下了。
- `s/pingt/hoge/g`：你在咒语最后加了一个 `g`。这下魔法棒火力全开，把书里所有的 pingt 统统变成了 hoge。

✓ 正确答案

- `sed -e s/pingt/hoge/g test.txt`
- `sed s/pingt/hoge/g test.txt`

✎ 考试小秘籍

题目中只要出现「全て」或「全ての」（全部），在使用 `sed` 的选项里一定要勾选带 `g` 的那个。这是 LPI 考试中最经典的考点之一。

这一知识点主要考察 正規表現（正则表达式）中各元字符的具体含义，特别是 `.`（任意字符）和 `[^...]`（排除特定字符）的组合逻辑。

💡 知识点总结

在正则表达式中，`.` 匹配任意一个字符，而 `[^b]` 匹配除了 "b" 以外的任意一个字符。因此，「a.[^b]d」是一个固定长度为 4 个字符的模式。

🔍 模式详细拆解

我们要找的字符串必须严格符合以下四个位置的定义：

- 1. **a**：第一位必须是小写字母 "a"。
- 2. **.**：第二位可以是 任意一个字符（包括字母、数字、空格等）。
- 3. **[^b]**：第三位可以是 除了 "b" 以外的任意一个字符。
- 4. **d**：第四位必须是小写字母 "d"。

記号	説明	使用例（一致する文字）
.	任意の1文字	a.. aから始まる3文字 (aaa,abcなど)
*	直前の文字の0回以上の繰り返し	* 任意の文字列
[]	[] 内のいずれか1文字 []内では、以下の表現と併用可能 - 範囲指定 ^ 先頭にある場合は後続の文字以外	[abc] a,b,c のどれか1文字 [a-z] 小文字のアルファベット1文字 [^abc] a,b,c 以外のどれか1文字
^	行頭	^a aから始まる行
\$	行末	a\$ aで終わる行
\	次の1文字をエスケープ (通常の文字として処理)	* *という文字
+	直前の文字の1回以上の繰り返し 【拡張正規表現】	a+ aを1回以上繰り返す文字列 (a,aa,aaaなど)
?	直前の文字の0回もしくは1回の繰り返し 【拡張正規表現】	windows? windowsのsが0または1文字 (window,windows)
	左右いずれかの文字列 【拡張正規表現】	abc xyz abcまたはxyz

选项逐一检查

- **baad**：正确答案。
 - 匹配过程：**a** (第1位) + **a** (任意字符) + **a** (不是b) + **d** (第4位)。完全吻合。
- **abdd**：正确答案。
 - 匹配过程：**a** (第1位) + **b** (任意字符) + **d** (不是b) + **d** (第4位)。完全吻合。
- **aacd**：正确答案。
 - 匹配过程：**a** (第1位) + **a** (任意字符) + **c** (不是b) + **d** (第4位)。完全吻合。
- **abd**：错误。
 - 理由：只有 3 个字符，而模式要求必须是 4 个字符。
- **abcd**：错误。
 - 理由：第三位是 "c"，看起来没问题，但要注意它的第三位匹配的是 **[^b]**。等等，重新看：
a (1) **b** (2) **c** (3) **d** (4)。
 - 修正：**a** (1) OK, **b** (2为 **.**) OK, **c** (3为 **[^b]**) OK, **d** (4) OK。所以 **abcd** 也是正确答案。

正規表現元字符背诵表

为了帮你像小学生一样记牢，请记住这些“变身”符号：

符号	含义 (日本語)	记忆口诀 (中文)
.	任意 の1文字	. 是一个点，点到谁就是谁。
[abc]	a, b, c の いずれか	括号里是选秀名单，中一个就行。
[^abc]	a, b, c 以外	^ 在括号里是“反骨”，偏不要这几个。
*	直前の文字の 0回以上 の繰り返し	* 是雪花，多点少点（甚至没有）都行。

🧠 小学生级记忆法：特工代码

想象你在破解一个 4 位数的特工密码 「a.[^b]d」：

1. 开头必须喊 “a”。
2. 第二个字母随便谁都行，哪怕是 “b” 也可以。
3. 第三个字母最关键，**绝对不能是 “b”**！只要不是 “b”，是谁都行。
4. 最后必须喊 “d” 结尾。

按照这个逻辑，只要长度是 4 且符合首尾和第三位的禁忌，就是正确答案。

✅ 正确答案

baad, abdd, aacd, abcd

(注：只要满足 a + 任意 + 非b + d 的四位组合即可)

📝 考试小秘籍

LPI 考试中，`[^...]` 的 `^` 符号最容易出错。记住：

- `^` 在 `[]` 里面：表示“排除/除了这个”。
- `^` 在 `[]` 外面（如 `^abc`）：表示“必须以这个开头”。

这一知识点主要考察 **正規表現（正则表达式）** 中关于 **文字クラス（字符类）** 的范围指定方法，特别是针对 **大文字（大写字母）** 的匹配规则。

💡 知识点总结：通配符 vs 正则表达式

通配符 (ワイルドカード/メタキャラクタ) 主要由 **シェル (Shell)** 处理，用于匹配 **文件名**；而 **正则表达式** (正規表現) 主要由 **特定命令** (如 `grep`, `sed`) 处理，用于匹配 **文件内的文本内容**。

🔍 深度对比：为什么它们不一样？

我们将通过**通配符** 和 **正则表达式** 来进行对比。

1.1 核心差异对照表

比较维度	通配符 (ワイルドカード)	正则表达式 (正規表現)
主要用途	查找文件 (如 <code>ls *.txt</code>)	查找文字 (如 <code>grep 'a*' test.txt</code>)
. (点)	代表当前目录	代表 任意的1文字 (任何一个字符)
* (星号)	代表 0文字以上 的任何字符	代表 直前の文字 (前一个字符) 重复 0 次以上
^ (尖角)	(无特殊含义, 除非在 [] 内代表排除)	代表 行頭 (行首)
处理者	シェル (Bash 等)	grep, sed, awk 等工具

1.2 最容易搞混的符号：* (星号)

这是考试中最致命的陷阱！

- 在通配符中：`ls a*` 意味着找 “以 a 开头的文件”。
- 在正则表达式中：`grep 'ba*'` 意味着找 “b 后面跟着 0 个或多个 a” 的内容（比如匹配 b, ba, baa）。

小学生级记忆法：警察与翻译官

想象你要在一栋大楼里找东西：

- 通配符 (ワイルドカード) 是 “大楼管理员”：
 - 你问他：“我想找名字里带 ‘张’ 字的文件柜。”
 - 他只看 柜子外面的标签 (文件名)。
 - 关键词：* 就是 “管它后面是什么”。
- 正则表达式 (正規表現) 是 “翻译官/侦探”：
 - 他打开柜子，一张张看里面的 信件内容 (文本)。
 - 他的规则更细：^ 是看信的第一行，\$ 是看信的结尾。
 - 关键词：* 是 “复读机”，重复前面的动作。

考试避坑：如何一眼区分题目考哪个？

看题目里给出的 コマンド (命令)：

- 如果是 `ls` , `cp` , `mv` , `rm` $\rightarrow$ 考的是 通配符 (ワイルドカード)。
- 如果是 `grep` , `sed` , `awk` , `vi` (的查找功能) $\rightarrow$ 考的是 正则表达式 (正規表現)。

举个例子 (实战演练)

假设文件夹里有文件：`a.txt`，`ab.txt`，`abb.txt`

1. 执行 `ls ab*` (通配符):

- 结果: `ab.txt`，`abb.txt` (匹配以 `ab` 开头的所有文件名)。

2. 执行 `grep "ab*" file` (正则表达式):

- 内容里有 `a`，`ab`，`abb` 都会被抓出来 (匹配 `a` 后面跟着 0 个或多个 `b`)。

知识点总结

在正则表达式中，使用 **角括弧** `[]` 来定义一个字符集合。如果要表示一个连续的范围，必须使用 **ハイフン** `-` 连接起始和结束字符。匹配 **大文字 (大写字母)** 的标准写法是 `[A-Z]`。

选项解析与详细说明

- `[A-Z]`：正确答案。
 - 含义：匹配从 `A` 到 `Z` 之间的 **任意一个** 大写字母。
 - 规范：在 `[]` 中使用 `-` 是表示 **範圍 (范围)** 的标准语法。
- `[^a-z]`：错误。
 - 理由：虽然 `^` 在 `[]` 中表示 **否定 (排除)**，但 `[^a-z]` 匹配的是“除了小写字母以外的 **任何字符**”。这包括了大写字母，但也包括了数字、标点符号、空格甚至换行符，不符合“大写字母1文字”的精确要求。
- `[a-z]`：错误。
 - 理由：这匹配的是 **小文字 (小写字母)**。
- `[A~Z]` 和 `[a~z]`：错误。
 - 理由：正则表达式中使用 **波浪号** `~` 并不表示范围。这会被解释为匹配“`A`”、“`~`”或“`Z`”这三个字符中的某一个。

正規表現范围指定对比表

为了帮你像小学生一样记牢，请记住这些“范围”的写法：

匹配目标 (日本語)	正则表达式	记忆口诀 (中文)
英大文字	[A-Z]	A 到 Z，用 横杠 连。
英小文字	[a-z]	小写的 a 到 z。
数字	[0-9]	从 0 数到 9。
英数字	[a-zA-Z0-9]	大小写加数字，全部装进筐。

🧠 小学生级记忆法：字母滑滑梯

想象字母表是一个长长的 滑滑梯：

1. `[A-Z]`：你坐在滑滑梯的顶端 **A**，顺着 横杠 `-` 一直滑到了底端 **Z**。这个过程覆盖了所有的大写字母。
2. 千万别用 `~`：滑滑梯必须是直的（横杠 `-`），如果用波浪号 `~`，滑滑梯就断了，你只能坐在 A、~ 或者 Z 这三个孤零零的点上。

✅ 正确答案

`[A-Z]`

📝 考试小秘籍

在 LPI 考试中，要注意区分 **大文字** 和 **小文字**。

- 如果题目要求“不区分大小写”，通常在命令后加 `-i` 选项（如 `grep -i`）。
- 如果在正则里想同时匹配大小写，写法是 `[a-zA-Z]`。

这一知识点主要考察 **viエディタ (vi编辑器)** 中有关 **行番号 (行号)** 显示设置的末行模式命令。

viの設定に関する主なviコマンド

コマンド	説明
<code>:set nu number</code>	行番号を表示
<code>:set nonu nonumber</code>	行番号を表示しない
<code>:set ts=タブ幅 tabstop=タブ幅</code>	タブ幅を数値で指定

💡 知识点总结

在 vi 的末行模式中，可以使用 `:set` 命令来控制各种环境变量。对于行号显示，可以使用 `number`（显示）或其缩写 `nu`，而要取消显示，则需在命令前加上 `no`。

🔍 选项解析与详细说明

根据 vi 的命令规范，要实现“非显示”行号，以下是正确的表达方式：

- `:set nonu`：正确答案。
 - 含义：`nu` 是 `number` 的缩写。`nonu` 即 **no number**，是执行频率最高的简写指令。
- `:set nonumber`：正确答案。
 - 含义：这是 `nonu` 的全拼形式，语义非常直观。
- `:set nonum`：错误。
 - 理由：虽然 `num` 看起来像缩写，但在 vi 的标准命令中，行号的正式缩写是 `nu`，因此 `nonum` 无法被识别。
- `:nonu`：错误。
 - 理由：修改环境设置必须以 `:set` 开头，直接输入 `:nonu` 会被当作无效命令。
- `:nonumber`：错误。
 - 理由：同上，缺少了关键的动词 `:set`。

📊 vi 行号显示/隐藏指令对照表

为了帮你像小学生一样记牢，请记住这两对“开关”：

目标动作 (日本語)	全拼指令	常用缩写指令	记忆口诀 (中文)
行番号を表示する	<code>:set number</code>	<code>:set nu</code>	nu 是 nu mber，给行穿上数字衣服。
行番号を非表示にする	<code>:set nonumber</code>	<code>:set nonu</code>	no 是拒绝，脱掉数字衣服。

🧠 小学生级记忆法：设置开关

想象 `set` 是一个“开关箱”：

1. 你想让 vi 做点什么设置，首先得喊一声 “`:set`” 拍一下开关箱。
2. 如果你想关掉某个功能，就在功能名字前面加个 “`no`”（不）。
3. 所以，“不（no）要数字（nu/number）” 的组合就是 `:set nonu` 或 `:set nonumber`。

✅ 正确答案

`:set nonu, :set nonumber`

📝 考试小秘籍

在 LPI 考试中，很多 `set` 相关的命令都有“全拼”和“缩写”两种正确形式（如 `number` 和 `nu`）。题目要求“**全て選択**”时，一定要把这两种形态都选上。

真的太抱歉了！刚才因为看到你题目里涉及 `tabstop` 这种 Linux 考试的专有名词，大脑自动切换到了“考试模式”，结果顺手就写成了日语。

咱们立刻换回中文，严格执行你的学习规则：**首句总结知识点，题目中文解析，关键术语保留日汉对照。**

💡 知识点总结：vi 文本缩进设置

在 **viエディタ**（vi编辑器）中，控制 **タブの幅**（制表符宽度）的环境变量是 `tabstop`，其对应的省略形式为 `ts`。

🔍 选项解析与详细说明（中文版）

这道题主要考查你对 `set` 命令中“全称”与“缩写”一一对应关系的记忆。

- `:set tabstop=10`：正确答案。
 - **理由**：这是标准的完整指令。`tabstop` 决定了按下 Tab 键时，光标跳跃的空格数。
- `:set ts=10`：正确答案。
 - **理由**：`ts` 是 `tabstop` 的法定缩写。在 Linux 考试（如 LPI）中，全称和缩写通常都会作为正确选项出现。
- `:set ts 10`：错误。
 - **理由**：语法错误。在 `set` 给变量赋值时，变量名与数值之间必须使用 `=`（等号）。
- `:set tab=10`：错误。
 - **理由**：vi 中并没有 `tab` 这个环境变量。

- `:set number=10`：错误。
 - 理由：`number`（缩写为 `nu`）是用来开关 行番号（行号）显示的，它不接受数值参数来改变宽度。

vi 常用环境设置对照表

为了帮你像小学生一样记牢，请记住这两对最重要的“双胞胎”：

设置目标 (日本語)	全称指令	缩写指令	记忆口诀
设置标签宽度	<code>:set tabstop=X</code>	<code>:set ts=X</code>	Tab Stop (TS) 停在第X格。
显示行号	<code>:set number</code>	<code>:set nu</code>	Number (NU) 给每行标号。

小学生级记忆法：小火车的车站

想象你在 vi 里打字，**Tab 键** 就像一辆 **小火车**：

1. `tabstop`：就是给小火车设定的“**车站 (Stop)**”。
2. `ts=10`：你告诉小火车：“每跑 **10** 步就设一个站。”
3. 记住等号：下命令时要用等号 `=`，就像给车站挂上“10号站”的牌子一样。

正确答案

- `:set tabstop=10`
- `:set ts=10`

考试小秘籍

在 LPI 考试中，凡是涉及到 `vi` 的环境设定，务必检查：

1. 有没有 `:set` 开头。
2. 赋值有没有用 `=`。
3. 缩写是否准确（如 `ts` 对应 `tabstop`，`nu` 对应 `number`）。

这一知识点主要考察 `sort` コマンド（排序命令） 中关于 `フィールド`（字段/列） 的指定、`区切り文字`（分隔符） 的设置以及 `数値`（数值） 排序选项的用法。

`sort` [オプション] [ファイル名]

オプション	説明
<code>-t</code> 区切り文字	指定した文字を区切り文字としてフィールドを認識
<code>-n</code>	数字を文字でなく数値としてソート
<code>-k</code> フィールド	ソート対象とするフィールドを指定（デフォルトは1つ目のフィールド）

💡 知识点总结

当需要对具有特定格式（如 CSV 格式）的文件进行排序时，必须使用 `-t` 指定分隔符，使用 `-k` 指定排序的列，并使用 `-n` 确保按数值大小而非字母顺序进行 `ソート`（排序）。

🔍 选项解析与详细说明

根据文件内容（如 `aaa, 200, AAA`），我们可以分析出正确命令的构成：

- `sort -k 2 -n -t , file`：正确答案。
 - `-k 2`：指定以 **2番目（第二个）** 字段为基准进行排序。
 - `-n`：指定按 **数値（数值）** 大小排序。如果不加 `-n`，1000 会排在 200 前面（因为 1 比 2 小）。
 - `-t ,`：指定 **区切り文字（分隔符）** 为逗号 `,`。只有这样，`sort` 才能正确识别出第二个字段是数字部分。
- `sort -k 2 -t , file`：错误。
 - **理由**：虽然指定了字段和分隔符，但缺少 `-n`。这会导致系统按 **文字（字符）** 顺序排列数字，结果会变成 `1000, 200, 300, 400`（按首位数字排序）。
- `sort -k -n , -t 2 file`：错误。
 - **理由**：选项后的参数对应错误。`-t` 后面应该接分隔符 `,`，而不是字段号 `2`。
- `sort -k , -n 2 -t file`：错误。
 - **理由**：语法混乱。`-k` 后面应接字段号，`-t` 后面应接分隔符。

📊 sort 命令关键选项对照表

为了帮你像小学生一样记牢，请记住这些字母的“职责”：

选项 (日本語)	作用 (中文解释)	记忆口诀
-k [数字]	キー (Key) となる列を指定	Key, 开启排序大门的钥匙 (哪一列)。
-t [字符]	区切り文字 (Separator) を指定	Tag, 给列与列之间打个标记。
-n	数値 (Numeric) としてソート	Number, 把内容当成数字看。
-r	逆順 (Reverse) にソート	Reverse, 反过来排。

小学生级记忆法：超市排队

想象文件里的数据是在超市排队的一群人：

- t , ：你发现每个人手里都拿着一个 逗号 (,) 做的隔板，把自己的名字和钱数分开了。
- k 2 ：你对保安说：“看他们手里拿的 第 2 样东西（钱数）。”
- n ：你强调说：“按 钱的多少（数值） 排队，钱多的往后站，别按名字字母排！”

✅ 正确答案

sort -k 2 -n -t , file

📖 考试小秘籍

在 LPI 考试中，-k、-n、-t 经常组合出现。最容易混淆的是 -t。记住：t 后面跟着的是“符号”，k 后面跟着的是“数字”。

这一知识点主要考察 リダイレクト（重定向）与 パイプ（管道） 的组合使用，以及如何通过 sort 和 uniq 命令处理文本。

💡 知识点总结

在 Linux 中，要合并、排序并去重，最标准的做法是先用 sort 对文件进行排序（uniq 只能处理连续的重复行），然后通过 パイプ | 传递给 uniq 命令，最后使用 追記リダイレクト >> 将结果保存。

🔍 选项解析与详细说明

- sort File* | uniq >> List ：正确答案。
 - sort File* ：将所有以 "File" 开头的文件内容合并并进行 並び替え（排序）。

- `| uniq`：通过管道接收排序后的结果，并删除 **重複行（重复行）**。注意：`uniq` 要求相同行必须相邻，所以必须先 `sort`。
- `>> List`：将最终结果 **追記（追加）** 到名为 "List" 的文件中。
- `tee File* | uniq > List`：错误。
 - **理由**：`tee` 命令用于将标准输入同时输出到屏幕和文件，它不具备排序功能，且这里用法不对。
- `expand File* | uniq << List`：错误。
 - **理由**：`expand` 是将制表符（Tab）转换为长度相等的空格；`<<` 是 **ヒアドキュメント**，用于输入多行文本，不是用来输出文件的。
- `sort File* | cut > List`：错误。
 - **理由**：`cut` 用于提取行的某一部分（字段），不能去除重复行；`>` 是覆盖重定向，不符合题目要求的「**追記**」。
- `cut File* | unexpand >> List`：错误。
 - **理由**：`unexpand` 是将空格转回制表符，与去重和排序无关。

文本处理常用命令对照表

为了帮你像小学生一样记牢，请记住这三个“加工厂”：

工具 (日本語)	核心功能 (中文解释)	记忆口诀
sort	並び替え (排序)	乱七八糟排排队。
uniq	重複を除く (去重)	相同的只留一个。
>>	追記 (追加)	两个箭头，在后面排队。

小学生级记忆法：做水果拼盘

想象你在做水果拼盘：

1. `sort File*`：你把 FileA 和 FileB 里的水果全部倒出来，按种类 **排好序**（苹果归苹果，梨归梨）。
2. `| uniq`：你看着那一排苹果，觉得太多了，**把重复的拿走**，每样只留一个。
3. `>> List`：你把处理好的水果 **放进（追加）** 已经有东西的篮子 `List` 里。

✅ 正确答案

`sort File | uniq >> List*`

📝 考试小秘籍

在 LPI 考试中，「重複を除き」（去除重复）这个词几乎总是和 `sort` 连用的。因为 `uniq` 比较“笨”，它只能发现挨在一起的重复行。如果不先 `sort`，它就没法彻底去重。

这一知识点主要考察用于对文件内容进行 **行单位（以行为单位）** 重新排列的 **ソート（排序）** 命令。

💡 知识点总结

在 Linux 中，`sort` 命令专门用于对文本文件的行进行排序。默认情况下，它会按照 **アルファベット順（字母顺序）** 或数字从小到大的顺序排列。

🔍 选项解析与详细说明

- `sort`：正确答案。
 - **功能**：行単位でファイルの内容をソートする（按行对文件内容进行排序）。它非常灵活，可以通过选项指定按数值排序（`-n`）、按特定列排序（`-k`）或逆序排序（`-r`）。
 - `fmt`：错误。
 - **功能**：テキストを整形する（格式化文本）。主要用于调整行的宽度（最大字符数），而不是改变行的先后顺序。
 - `head`：错误。
 - **功能**：ファイルの先頭部分を表示する（显示文件的开头部分）。默认显示前 10 行。
 - `cut`：错误。
 - **功能**：行から特定の部分を取り出す（从行中提取特定部分）。例如提取每一行的第 1 到第 5 个字符。
 - `nl`：错误。
 - **功能**：行番号を付加する（添加行号）。
-

📊 常用文本处理命令对比表

为了帮你像小学生一样记牢，请记住这些命令的“动作”：

命令	动作 (日本語)	记忆口诀 (中文)
sort	行を 並べ替える	Sort (排序)，像排队一样排整齐。
head	先頭 を表示	Head (头)，只看最前面的部分。
cut	部分を 切り出す	Cut (切)，把一行里不需要的部分切掉。
nl	行番号 をつける	Number Lines (行号)，给每一行标上号。

🧠 小学生级记忆法：整理书架

想象你有一堆乱七八糟的书（文件行）：

- 1. `sort`：就像一个**整理员**。他把所有的书按照书名的首字母 A 到 Z 重新摆放。
- 2. `head`：就像一个**挑剔的读者**。他只看书架最上面那层的书，剩下的不看。
- 3. `cut`：就像一个**手工老师**。他把每本书的封面剪掉一半，只留下标题。

✅ 正确答案

sort

📝 考试小秘籍

题目中只要看到「ソート」或「並べ替え」，想都不用想，答案就是 `sort`。如果你想把排序后的结果去重，通常会配合 `uniq` 命令一起使用。

这一知识点主要考察 `tail` コマンド（显示文件尾部命令）中关于指定 **行数（行数）** 的参数格式。

tail [オプション] [ファイル名]

オプション	説明
-n 行数 -行数	指定した行数をファイルの 末尾から表示
-c バイト数	指定したバイト数をファイルの 末尾から表示
-f	ファイルの末尾に追加された 行を表示し続ける

💡 知识点总结

在 Linux 中，`tail` 命令默认显示文件的最后 10 行。若要指定显示 **末尾 5 行**，最标准的做法是使用 `-n 5`，但在许多版本中也支持简写形式 `-5`。

🔍 选项解析与详细说明

- `tail -n 5 httpd.conf`：正确答案。
 - 含义：`-n` (number of lines) 是指定行数的标准选项。后面跟着数字 5，表示显示最后 5 行。
- `tail -5 httpd.conf`：正确答案。
 - 含义：这是 `-n 5` 的快捷简写形式。虽然在某些极严格的 POSIX 环境下建议用 `-n`，但在大多数 Linux 系统（如 LPI 考试涉及的环境）中，直接跟数字是完全有效的。
- `tail -c 5 httpd.conf`：错误。
 - 理由：`-c` (bytes) 代表 **バイト数（字节数）**。这个命令会显示文件的最后 5 个字节（通常只有几个字母或换行符），而不是 5 行。
- `tail -l 5 httpd.conf`：错误。
 - 理由：`tail` 命令并没有 `-l` 这个选项。可能容易与 `pr -l`（指定页面长度）搞混。
- `tail 5 httpd.conf`：错误。
 - 理由：缺少连字符 `-`。如果没有连字符，系统会认为你想打开一个名为 "5" 的文件和 "httpd.conf" 文件。

📊 head 与 tail 常用选项对比表

为了帮你像小学生一样记牢，请记住这对“头尾”兄弟：

命令	默认动作 (日本語)	指定 5 行的写法	记忆口诀
head	先頭 10行を表示	head -n 5	Head 是头，看最上面。
tail	末尾 10行を表示	tail -n 5	Tail 是尾巴，看最下面。

🧠 小学生级记忆法：看风筝

想象整个文件是一只长长的 **风筝**：

1. `head`：是看风筝的 **头**。
2. `tail`：是看风筝的 **尾巴**。
3. `-n 5`：是你拿着放大镜，只想看尾巴最后那 **5 厘米（5 行）** 的花纹。

✅ 正确答案

`tail -n 5 httpd.conf, tail -5 httpd.conf`

考试小秘籍

在 LPI 考试中，要注意 `-n` 和 `-c` 的区别。

- `n` = number (行数)
- `c` = character / bytes (字符/字节数)
- 另外，`tail -f` 是另一个超级高频考点，用于「リアルタイムで監視」（实时监控）日志文件的更新。

这一知识点主要考察 **viエディタ（vi编辑器）** 的启动行为及其 **初期モード（初始模式）** 的状态。

知识点总结

当使用 `vi ファイル名` 启动时，如果该文件不存在，vi 会以该文件名创建一个 **新規ファイル（新文件）** 的编辑缓冲区，并默认进入 **コマンドモード（命令模式）**。

选项解析与详细说明

- 「test.txt」という空のファイルが開き、コマンドモードになる：正确答案。
 - **文件处理**：即使 `test.txt` 不存在，vi 也不会报错。它会打开一个空的缓冲区（Buffer），并在屏幕下方提示 `[New File]`（新文件）。
 - **初始模式**：vi 启动后的默认状态始终是 **コマンドモード（命令模式）**。此时你不能直接打字，需要输入 `i` 或 `a` 等指令切换到 **入力モード（输入模式）** 才能开始编辑。
- 無名の空のファイルが開き...：错误。
 - **理由**：因为启动命令里已经指定了 `test.txt`，所以这个缓冲区是有名字的。只有直接输入 `vi`（不带文件名）启动时，才会是无名状态。
- 「test.txt」という空のファイルが開き、入力モードになる：错误。
 - **理由**：vi 绝对不会在启动时直接进入输入模式。这常是初学者的误区，导致一上来打字却触发了一堆命令。
- ファイルが存在しないのでエラーになる：错误。
 - **理由**：在 Linux 中，很多编辑器（如 vi, nano）在打开不存在的文件时都会自动进入“新建”流程。

vi 启动后的状态背诵表

为了帮你像小学生一样记牢，请记住这个“开门”的状态：

状态属性	内容 (日本語)	记忆口诀 (中文)
ファイル名	指定した名前 (test.txt)	带着名字出生。
初期モード	コマンドモード	Command 先行，先发令再干活。
ファイル内容	空 (Empty)	白纸一张。
画面下方の表示	[New File]	欢迎新同学。

🧠 小学生级记忆法：走进教室

想象你（用户）走进一间叫 `test.txt` 的新教室（vi）：

1. **你是班长**：你一进门，默认身份是 “**指挥官（命令模式）**”。你不能直接在黑板上写字，你得先下个口令。
2. **变身口令**：你喊一声 `i`（Insert），你才变成了 “**学生（输入模式）**”，这时候你才能在黑板上写写画画。
3. **没教室怎么办？**：如果没有这间教室，学校会现场给你盖一间新的，绝对不会把你关在门外（报错）。

✅ 正确答案

「test.txt」という空のファイルが開き、コマンドモードになる

📝 考试小秘籍

这是 LPI 考试中最基础的送分题。只要记住：

- 启动 vi = **命令模式**
- 退出 vi = `:q` 系列

这一知识点主要考察 **リダイレクト（重定向）** 的基本符号及其功能区别，特别是 **上書き（覆盖）** 与 **追記（追加）** 的不同。

💡 知识点总结

在 Linux 中，使用 `>` 符号可以将命令的 **標準出力（标准输出）** 保存到文件中。如果文件已存在，其内容会被 **上書き（覆盖）** 掉。

选项解析与详细说明

- `date > log.txt`：正确答案。
 - 功能：这是最常用的 **出力リダイレクト（输出重定向）** 符号。它会将 `date` 的结果写入 `log.txt`，并删除该文件原有的所有内容（上書き）。
- `date 1> log.txt`：正确答案。
 - 功能：数字 `1` 代表 **標準出力 (stdout)**。实际上 `>` 只是 `1>` 的缩写形式，两者的效果完全相同。
- `date >> log.txt`：错误。
 - 理由：`>>` 是 **追記（追加）** 符号。它会将新内容添加到文件末尾，而不会删除原有内容。题目要求的是“上書き保存”。
- `date 2> log.txt`：错误。
 - 理由：数字 `2` 代表 **標準エラー出力 (stderr)**。由于 `date` 命令正常执行时产生的是标准输出而不是错误，这个命令会导致 `log.txt` 变成一个空文件（或者保留原样），而日期信息会直接显示在屏幕上。
- `date < log.txt`：错误。
 - 理由：`<` 是 **入力リダイレクト（输入重定向）**。它尝试将 `log.txt` 的内容作为 `date` 命令的输入，这不符合逻辑，也不会保存任何结果。

重定向符号背诵表

为了帮你像小学生一样记牢，请记住这些“漏斗”的方向：

符号 (日本語)	作用 (中文解释)	记忆口诀
<code>></code> 或 <code>1></code>	標準出力を 上書き	一个箭头，力气大，把旧的都冲走。
<code>>></code>	標準出力を 追記	两个箭头，比较温柔，排在旧的后面。
<code>2></code>	標準エラー出力を保存	2 是错误（Error）的代号。
<code><</code>	標準入力として使う	箭头向左，把文件的东西喂给命令。

小学生级记忆法：倒水实验

想象你在往一个杯子（文件）里倒水（数据）：

1. `>` (**上書き**)：就像你先在大喊：“通通倒掉！”把杯子里的水倒空，然后再灌入新水。
2. `>>` (**追記**)：就像你小心翼翼地接着往里倒，不管原来有多少，都在后面接着加。

3. 题目要求：是要“上書き”，所以选那个带一个箭头的 `>`（或者带标准输出编号的 `1>`）。

✅ 正确答案

- `date > log.txt`
- `date 1> log.txt`

📝 考试小秘籍

LPI 考试经常考查数字 **1** 和 **2** 的含义：

- **1 = stdout** (正常输出)
- **2 = stderr** (报错信息)
- 如果题目问如何把错误和正常输出一起保存，记得那个组合技：`&>`。

关于 **文本处理选项** 和 **环境变量** 的核心知识点

環境変数	説明
HISTFILE	コマンド履歴保存ファイルのパス
HISTSIZE	現在のシェルでのコマンド履歴の保存数
HISTFILESIZE	コマンド履歴保存ファイルへの履歴保存数
HOSTNAME	ホスト名
HOME	ログインしているユーザのホームディレクトリ
LANG	ロケール(言語設定)
PATH	コマンドやプログラムを検索するディレクトリのリスト
PWD	カレントディレクトリのパス
USER	ログインしているユーザ

💡 核心知识点总结

1.3 行号显示： `cat` 与 `nl` 的选项映射

这是最常考的细节。你需要记住哪些选项是等价的，以及它们对 **空行（空白行）** 的处理方式。

功能需求 (中文)	cat 命令选项	nl 命令选项	对应说明 (日语)
包含空行，给所有行编号	<code>-n</code>	<code>-b a</code>	空行を含めた全ての行に行番号をつける
排除空行，仅给有内容行编号	<code>-b</code>	<code>-b t</code> (默认)	空行を除いた行に行番号をつける

1.2 核心环境变量 (環境変数)

环境变量是 Shell 用来存储系统配置信息的变量。

- **PATH** : 最重要的变量。它是系统搜索命令或可执行程序的路径列表。
 - **HOME** : 当前登录用户的家目录路径。
 - **PWD** : 当前所在的工作目录（Current Directory）路径。
 - **HISTSIZE** : 当前 Shell 能够保存的命令历史记录条数。
 - **LANG** : 系统的语言和本地化设置（Locale）。
-

重点避坑指南

- **sort** 命令的数值陷阱:
 - 在处理你提供的 **file** 文件（包含数字 200, 1000 等）时，如果不加 **-n**，排序会按字符进行。结果 1000 会排在 200 前面，因为 "1" 比 "2" 的 ASCII 码小。数值排序必须用 **-n**。
 - **nl** 的默认行为:
 - 记住 **nl** 不加任何选项时，默认就是 **-b t**（即不给空行编号）。
-

记忆小技巧：字母关联法

- **n** = Number All（全部编号）
- **b** = Body Only（只给有内容的“身体”编号）
- **a** = All（全部）
- **t** = Text（仅文本内容）

这一知识点主要考察的是 **cd** (Change Directory) 命令在切换目录时的路径简写规则，特别是 **チルダ (~)** 在不同情况下的含义。

知识点总结

当你在命令中指定 **~用户名** 时，无论当前登录的是谁，系统都会跳转到该 **指定ユーザ（指定用户）** 的 **ホームディレクトリ（家目录）**。

选项解析与详细说明

- **cd ~test** : 正确答案。
 - **理由**：在 Linux 中，**~用户名** 这种格式表示该用户的家目录。即使你当前是 **root** 用户，输入 **cd ~test** 也会直接让你跳转到 **/home/test**。

- `cd`：错误。
 - **理由**：不带参数的 `cd` 会回到 **当前登录用户** 的家目录。因为你现在是 **root**，所以会跳到 `/root`，而不是 `test` 用户的家目录。
- `cd ~`：错误。
 - **理由**：`~` 代表的是 **当前用户**（即 `root`）的家目录 `/root`。
- `cd ~/test`：错误。
 - **理由**：这表示进入 **当前用户家目录（/root）** 下一个名叫 `test` 的子目录，即 `/root/test`。
- `cd test`：错误。
 - **理由**：当前在 `/etc` 目录下，这会尝试进入 `/etc/test`，这显然不是 `test` 用户的家目录。



(チルダ) 路径规则对照表

为了帮你像小学生一样记牢，请记住这三个“波浪号”的用法：

表达方式	跳转目的地 (日本語)	目标路径 (示例)
~ (单独使用)	現在のユーザ のホーム	/root (如果你是 root)
~用户名	そのユーザ のホーム	/home/test
~/目录名	自分のホーム下 のディレクトリ	/root/某个目录



小学生级记忆法：找邻居的家

想象你住在一个叫“家目录大楼”的地方：

1. `cd ~`：你对自己说：“我要回家。” 于是你回到了 **你自己 (root)** 的房间。
2. `cd ~test`：你对自己说：“我要去 **test** 家。” 于是你顺着门牌号找到了 `test` 的房间。
3. **题目要求**：你是 `root`，但想去 `test` 家，所以必须大声喊出对方的名字：`~test`。



正确答案

`cd ~test`

这是 LPI 考试中关于 “权限与路径” 的经典陷阱。

- 如果当前用户是 **test**，那么 `cd`、`cd ~`、`cd ~test` 三者等价。
- 但如果当前用户是 **root**，只有 `cd ~test` 能准确带你去 test 的家。

这一知识点主要考察不同操作系统中 **改行コード（换行符）** 的标准差异，这是进行跨平台开发和脚本编写时必须掌握的基础。

💡 知识点总结

改行コード 是表示一行结束的特殊字符。**Linux** 统一使用 **LF**（Line Feed），而 **Windows** 为了兼容古老的打印机标准，使用的是 **CR**（Carriage Return）与 **LF** 的组合，即 **CRLF**。

🔍 选项解析与详细说明

- **Linux: LF (\n)、Windows: CRLF (\r\n)：正确答案。**
 - **Linux (Unix/macOS):** 采用 **LF**。在代码中表示为 `\n`。
 - **Windows:** 采用 **CRLF**。由两个动作组成：回到行首（CR, `\r`）和换到下一行（LF, `\n`）。
- **其他选项：**均为错误组合。
 - **CR (\r)：**这曾是旧版 Mac (Classic Mac OS) 使用的标准，现在已基本被 LF 取代。
 - 如果在 Linux 下打开 Windows 创建的文本，经常会在行尾看到奇怪的 `^M` 符号，那其实就是多出来的 **CR (\r)**。

📊 各系统改行コード对照表

为了帮你像小学生一样记牢，请参考下表：

操作系统 (日本語)	改行コード名称	字符表示	含义
Linux / Unix	LF	\n	Line Feed (换行)
Windows	CRLF	\r\n	Carriage Return + Line Feed (回车+换行)
旧 Mac (OS 9以前)	CR	\r	Carriage Return (回车)

🧠 小学生级记忆法：打字机的故事

想象你正在用一台古老的 **打字机**：

1. **Windows (CRLF)**: 你写完一行，先拨动手柄让色带头 **回到左边 (CR)**，再滚一下滚轮 **向下走一行 (LF)**。动作很多，所以是两个字母。
2. **Linux (LF)**: 这台打字机很先进，你只要按一下，它就自动帮你 **跳到下一行 (LF)** 的开头了。动作干练，只有一个字母。

✅ 正确答案

Linux: LF (\n) 、 Windows: CRLF (\r\n)

📝 考试小秘籍

在 LPI 考试中，经常会考到如何转换这两个格式。记住两个常用的转换命令工具：

- `tr -d '\r'`：删除掉所有的 CR，把 Windows 文件转成 Linux 格式。
- `dos2unix` 和 `unix2dos`：专门负责在两个系统间转换文件的改行コード。

这一知识点主要考察的是 **viエディタ (vi编辑器)** 的家族演变及其最著名的 **派生 (衍生)** 版本。

💡 知识点总结

Vim 的全称是 **Vi IMproved** (改进版 vi)，它是从传统的 vi 派生出来的功能更强大、支持语法高亮和多级撤销的高性能文本编辑器。

🔍 选项解析与详细说明

- **Vim: 正确答案。**
 - **含义: Vi IMproved。** 正如其名，它是 vi 的增强版。
 - **特点:** 增加了可视模式、支持多种编程语言的 **構文強調 (语法高亮)**、更强大的搜索替换功能以及无限次的 **アンドゥ (撤销)**。
- **vi: 错误。**
 - **理由:** 它是原型 (Original)，不是从自己派生出来的。
- **EDITOR: 错误。**
 - **理由:** 这是一个 **環境変数 (环境变量)** 的名称，用来指定系统默认使用的编辑器 (比如 `EDITOR=vi`)。
- **nano: 错误。**

- **理由：**它是一个简单易用的字符终端编辑器，但它属于 **Pico** 编辑器的克隆，与 vi 的操作逻辑完全不同。
- **Emacs：**错误。
 - **理由：**它是 Linux 世界中与 vi 并驾齐驱的另一大编辑器流派，但它拥有独立的发展历史，并非由 vi 派生。

编辑器家族简表

为了帮你像小学生一样记牢，请看这个简单的“族谱”：

编辑器	角色 (日本語)	关系说明 (中文)
vi	元祖 (Original)	所有的起点，最基础的版本。
Vim	派生 / 高機能	Vi IMproved ，穿了机甲的 vi。
Emacs	ライバル (对手)	另一大家族，功能多到像个操作系统。

小学生级记忆法：进化论

想象编辑器是在玩《宝可梦》进化：

1. **vi**：是初始形态的小火龙，只有最基本的喷火功能（基础编辑）。
2. **Vim**：是进化后的 **喷火龙**。它不仅会喷火，还会飞（窗口分割）、会各种强力招式（插件支持）。
3. **名字揭秘**：**Vim** 的名字里就藏着 **Vi**，后面加个 **m**（Improved，变强了）。

正确答案

Vim

考试小秘籍

在 LPI 考试中，虽然题目经常写 `vi`，但在现代 Linux 系统（如 CentOS, Ubuntu）中，当你输入 `vi` 时，系统运行的通常其实就是 **Vim**。

エディタ	説明
Emacs	テキスト編集、コーディング、デバッグ、他プログラムの操作など開発環境として便利な機能を備えた高機能エディタ ユーザ独自の拡張機能をカスタマイズでき、GUIもサポートしている
nano	GUIをサポートしていない軽量の端末エディタ 操作キーとそれに対応する機能が画面下部に表示されており、初心者でも使いやすい
Vim	viから派生した高機能なテキストエディタ 基本的な操作方法是viと同じで、GUIもサポートしている 現在はviに代わり、Linuxで標準的に使用されている

Linux 系统中最常见的文本编辑器（テキストエディタ）主要分为：以快捷键见长的 **Emacs**、以简单易用著称的 **nano**，以及由经典 vi 派生、功能最强大的 **Vim**。

🔍 三大编辑器详细对比（中文版）

我们将从**起源**、**操作特点**和**适用场景**来分析，帮你彻底分清它们。

编辑器	派生与背景 (日本語)	操作特点 (中文)	考试/实战地位
Vim	vi から派生した高機能版	模式切换（命令/输入/末行模式）。功能最强，学习曲线最陡。	必考核心。所有 Linux 工程师必须掌握。
nano	Pico から派生した単純なエディタ	所见即所得。底部直接显示快捷键（如 ^O 保存），非常直观。	基础考点。适合新手快速修改简单配置。
Emacs	独自の発展を遂げた高機能エディタ	组合键控制（如 Ctrl 或 Alt 组合）。它不仅是编辑器，还能发邮件、玩游戏。	常考选项。作为 Vim 的劲敌经常出现在干扰项中。

📁 重点解析：nano 的独特之处

相比于需要“变身”才能打字的 Vim，**nano** 的逻辑更像 Windows 的记事本：

1. **直接输入**：打开就能打字，不需要像 Vim 那样先按 **i** 进入 **入力モード（输入模式）**。
2. **底部菜单**：屏幕下方那一排带 **^** 符号的字母就是快捷键。
 - **^** 代表 **Ctrl** 键。
 - **^O (WriteOut)**：保存文件。
 - **^X (Exit)**：退出。
3. **简单但功能有限**：它没有 Vim 那么多复杂的正则匹配、窗口分割等 **高機能（高阶功能）**。

🧠 小学生级记忆法：三位“文字老师”

想象你要写一篇作文，请了三位老师来帮忙：

1. **Vim 老师**：“严厉的老学究”。你得先学会他的“手语（命令模式）”才能动笔，但他懂的知识（插件和功能）最多，效率最高。
 2. **nano 老师**：“亲切的幼儿园老师”。他把怎么保存、怎么退出都写在黑板底下了，你直接写就行，不用记复杂的规矩。
 3. **Emacs 老师**：“全能魔术师”。他不仅教你写作文，还能一边教你一边变出一台收音机听新闻，但他用的魔法咒语（快捷键组合）非常复杂。
-

✅ 考试避坑：识别“派生”关系

- 如果题目问“**vi 派生**”：选 **Vim**。
- 如果题目问“**最简单的模式**”：通常指 **nano**。
- 如果题目问“**EDITOR 环境变量**”：这三个都可以作为值填进去。

💡 知识点总结

nano 是一款以 **轻量（轻量）** 和 **直感的（直观）** 著称的文本编辑器，其最大的特点是在屏幕底部直接列出了 **操作キ一（操作键）** 及其对应的功能。

🔍 选项解析与详细说明

- **nano**：正确答案。
 - **操作界面**：在屏幕最下方显示如 `^G Get Help`、`^O Write Out` 等提示，其中 `^` 代表 **Ctrl 键**。
 - **易用性**：它是典型的“所见即所得”编辑器，打开后即可直接输入文字，无需像 vi 那样切换模式。
- **Vim / vi**：错误。
 - **理由**：vi 及其派生版本 Vim 采用 **モード切り替え（模式切换）** 机制。如果不熟悉命令模式，初学者甚至连如何输入文字和退出程序都找不到。屏幕下方通常只显示文件名或状态信息，不会常驻操作指南。
- **Emacs**：错误。
 - **理由**：虽然功能极其强大，但它依赖于复杂的组合键（如 `Ctrl+x` 接着 `Ctrl+c` 退出），且默认界面并不会在底部持续显示所有操作指令，学习曲线较陡。
- **EDITOR**：错误。

- 理由：这只是一个用于指定系统默认编辑器的 **環境変数（环境变量）** 名。

编辑器界面特征对比表

为了帮你像小学生一样记牢，请看这三种编辑器的“表情”：

编辑器	底部显示内容 (日本語)	操作逻辑 (中文)
nano	操作キーのヘルプ	看小抄：底部写着怎么存、怎么退。
Vim	モード名 / ファイル名	记暗号：必须记住 i 才能写，:wq 才能走。
Emacs	ミニバッファ (状態表示)	练钢琴：全靠复杂的手指组合键。

小学生级记忆法：底部的小抄

想象你在参加一场文本编辑考试：

- nano**：是一位超温柔的老师，他直接把 **答案（快捷键）** 印在考卷的 **页脚（底部）** 了。你只要看着底下的提示按 `Ctrl+O` 就能存盘，完全不用背。
- Vim**：是一位严厉的老师。你进考场发现卷子是封印的，你得先大喊一声 `i`（进入输入模式）封印才解开；交卷时还得输入神秘代码 `:wq`。

正确答案

nano

考试小秘籍

在 LPI 考试中，题目里只要提到「画面下部に表示」（在画面底部显示提示）或「使いやすい」（易于使用），基本可以锁定答案为 `nano`。

这一知识点主要考察 **リダイレクト（重定向）** 符号中关于 **標準入力（标准输入）** 的切换操作。

知识点总结

在 Linux 中，使用 `<` 符号可以将命令的输入来源从默认的 **キーボード（键盘）** 切换为指定的 **ファイル（文件）**。

选项解析与详细说明

- `<`：正确答案。

- **含义：入力リダイレクト（输入重定向）**。它告诉命令：“不要等我从键盘输入，去读这个文件里的内容吧。”
- `>`：错误。
 - **理由：出力リダイレクト（输出重定向）**。将命令的执行结果 **上書き（覆盖）** 保存到文件中。
- `>>`：错误。
 - **理由：追加リダイレクト（追加重定向）**。将结果 **追記（追加）** 到文件末尾。
- `2>`：错误。
 - **理由：標準エラー出力（标准错误输出）** 的重定向。将报错信息保存到文件。
- `2>>`：错误。
 - **理由**：将报错信息 **追加** 保存到文件。

重定向符号方向对照表

为了帮你像小学生一样记牢，请观察箭头的“指向”：

符号 (日本語)	数据流动方向	记忆口诀
命令 < 文件	文件 → 命令	嘴巴向左，把文件“吃”进命令里。
命令 > 文件	命令 → 文件	箭头向右，把结果“射”进文件里。
命令 >> 文件	命令 → 文件 (末尾)	两个箭头，力量叠加，排队往后加。

小学生级记忆法：喂食游戏

想象命令是一个 **小怪兽**：

1. `<` (**输入**)：小怪兽张开大嘴向左边看，它在吃掉你给它的 **文件饼干**。
2. `>` (**输出**)：小怪兽吃饱了，向右边 **吐出结果**，存进文件盒子里。
3. **题目要求**：要把键盘换成文件作为“来源”，就是让小怪兽去吃文件，所以选 `<`。

正确答案

`<`

考试小秘籍

在 LPI 考试中，`<` 经常出现在 `tr` 或 `wc` 命令的题目中。例如：

- `wc -l < test.txt`：统计文件行数（输入来自文件）。
- 注意：虽然很多命令可以直接加文件名（如 `cat file`），但了解 `<` 的原理对于理解 **ヒアドキュメント**（Here Document, `<<`）非常关键。

这一知识点主要考察 **正規表現**（正则表达式）中关于 **行頭**（`^`）、**行末**（`$`）以及 **任意文字列**（`.*`）的精准匹配规则。

```
$ cat test.txt
1
12
123
1234
12345
```

💡 知识点总结

要精确匹配某一行的完整内容，必须使用 `^` 表示该行开始，使用 `$` 表示该行结束，从而排除掉包含相同字符但长度不同的行（例如排除“1234”）。

🔍 选项解析与详细说明

根据你提供的 `test.txt` 文件内容：

- 第1行： `1`
- 第2行： `12`
- 第3行： `123` （目标行）
- 第4行： `1234`
- 第5行： `12345`

我们来逐一分析正则表达式：

- `^1.*3$`：正确答案。
 - `^1`：匹配以 1 开头的行。
 - `.*`：匹配中间任意数量的字符（在这里匹配的是数字 `2`）。
 - `3$`：匹配以 3 结尾的行。
 - **结果**：在文件中，只有 `123` 这一行既以 1 开头又以 3 结尾，因此精准匹配。
- `^[^2-3]23$`：正确答案。

- `[^2-3]`：这是一个 **否定字符类**，匹配“不是 2 且不是 3”的任意 **一个** 字符。在 `123` 中，第一个字符是 `1`，符合条件。
- `23$`：紧接着匹配 `23` 并且该行到此结束。
- **结果**：匹配 `123`。
- `^1.*3`：错误。
 - **理由**：虽然它匹配 `123`，但因为它没有结尾符号 `$`，它也会匹配 `1234` 和 `12345`（因为这两行也包含“以1开头且后面有3”的部分）。
- `[1-3]`：错误。
 - **理由**：这只代表“1、2、3 中的任意 **一个** 数字”。它会匹配每一行里的单个数字，而不是整行内容。
- `^13$`：错误。
 - **理由**：它要求行首是 1，紧接着就是结尾 3。它只能匹配内容恰好为 `13` 的行，而我们的目标行中间还有一个 `2`。

正規表現定位符对照表

为了帮你像小学生一样记牢，请记住这对“门神”：

符号 (日本語)	作用 (中文解释)	记忆口诀
<code>^</code>	行頭 (开始)	尖角向上，代表开始。
<code>\$</code>	行末 (结束)	美元符号，代表收尾。
<code>.</code> <code>*</code>	任意の文字列	点是任意字，星是变变变。

小学生级记忆法：排队点名

想象数字们在排队，老师要点名请 `123` 出来：

1. `^1...3$`：老师说：“我要找那个 **排头是 1，排尾是 3** 的小队。”
 - `1234` 想出来，但老师说：“不行，你的排尾是 4。”
 - `12` 想出来，但老师说：“不行，你的排尾是 2。”
 - 只有 `123` 完美符合。
2. `[^2-3]23$`：老师说：“找一个 **开头不是 2 也不是 3**，后面跟着 **23 就散伙（结束）** 的队伍。”只有 `123` 符合。

✔ 正确答案

- `*^1.3$`
- `[^2-3]23$`

📝 考试小秘籍

在 LPI 考试中，“精准匹配” 必须有 `^` 和 `$` 的双重保护。如果选项里少了其中一个，它通常会产生 “部分匹配” 的结果，从而导致多选了不该选的行。

这一知识点主要考察 **viエディタ（vi编辑器）** 在编辑过程中切换文件的 **末行模式（コマンドラインモード）** 指令。

コマンド	説明
<code>:q</code>	viを終了 ただし、内容が変更されている場合は、警告が出て終了できない
<code>:q!</code>	viを強制終了 内容が変更されている場合でも保存せず終了
<code>:w</code>	編集内容を保存
<code>:wq</code> <code>:x</code> <code>ZZ</code>	編集内容を保存しviを終了
<code>:wq!</code>	編集内容を保存しviを強制終了 読み取り専用のファイルでも強制的に保存して終了（強制保存に失敗したらviは終了しない）
<code>:e</code> ファイル名	viを終了せずに、現在開いているファイルを閉じて別ファイルを開く ただし、内容が変更されている場合は、警告が出て別ファイルを開けない
<code>:e!</code> [ファイル名]	viを終了せずに、現在開いているファイルを閉じて別ファイルを開く 内容が変更されている場合は、現在開いているファイルを最後に保存した状態に戻してから別ファイルを開く ファイル名を省略した場合 (<code>:e!</code> のみ) は、現在開いているファイルを最後に保存した状態に戻す

💡 知识点总结

在 vi 中，使用 `:e` (**edit**) 命令可以关闭当前缓冲区并打开一个 **別のファイル（另一个文件）**。如果当前文件有未保存的修改，需要使用 `!` 强制执行。

选项解析与详细说明

- `:e newfile`：正确答案。
 - 含义：`e` 是 **edit** 的缩写。这条命令用于在不退出 `vi` 的情况下打开新文件。
 - 前提：只有当当前文件 **没有未保存的修改** 时，此命令才有效。
- `:e! newfile`：正确答案。
 - 含义：这里的 **!** 表示 **强制执行**。
 - 功能：即使当前文件有修改且未保存，也会直接 **破棄（丢弃）** 那些修改，并强行打开 `newfile`。
- `:q! newfile`：错误。
 - 理由：`:q!` 会直接退出 `vi` 编辑器。它不接受文件名作为参数来打开新文件。
- `:ZZ newfile`：错误。
 - 理由：`ZZ` 是在 **コマンドモード（命令模式）** 下使用的快捷键，用于保存并退出。它不能带文件名参数，且通常不带冒号（除非在末行模式输入 `:wq`）。
- `:x newfile`：错误。
 - 理由：`:x` 的功能与 `:wq` 类似，即保存并退出。它同样不支持在退出时顺便打开另一个文件。

vi 文件切换与退出指令对比表

为了帮你像小学生一样记牢，请记住这些字母的“性格”：

命令	核心动作 (中文)	考试关键词 (日语)	记忆逻辑
<code>:q</code>	退出	終了、警告が出る	Quit（正常离开）
<code>:q!</code>	强制退出	強制終了、保存せず	不管三七二十一，! 直接走
<code>:w</code>	保存	保存	Write（写入硬盘）
<code>:wq / :x / ZZ</code>	保存并退出	保存し終了	先写 (W) 后走 (Q)
<code>:wq!</code>	强制保存退出	強制的に保存して終了	遇到只读文件也要硬写
<code>:e ファイル名</code>	切换文件	別ファイルを開く、警告が出る	Edit（换个文件编辑）
<code>:e! [ファイル名]</code>	强制切换/还原	保存した状態に戻す、別ファイル	弄乱了？用! 还原或强行换

小学生级记忆法：换课本

想象你正在 vi 这张 **课桌** 上写作业：

1. `:e newfile`：你对老师说：“这张卷子写完了，我要换 **新的 (e)** 卷子 `newfile`。”
2. `:e! newfile`：你对老师说：“这张卷子我写乱了，**不想要了 (!)**，直接给我换张 **新的 (e)**。”
3. 千万别选 `:q`：选了 `:q` 就像是你直接背起书包回家了，没法在桌上接着写新卷子。

针对考试关键词的深度拆解

1.3 关于“退出”的陷阱

- 关键词：警告が出る $\rightarrow$ 选 `:q` 或 `:e`。这代表你改了东西但没保存，系统拦着你。
- 关键词：保存せず $\rightarrow$ 必须选带 `!` 的，如 `:q!`。

1.2 关于“文件切换”

- 关键词：viを終了せずに（不退出vi） $\rightarrow$ 锁定 `:e` 系列。
- 关键词：状態に戻す（回到初始状态） $\rightarrow$ 锁定 `:e!`（不加文件名时就是重新读入当前文件）。

1.3 关于“保存退出”的三胞胎

- `:wq`，`:x`，`ZZ` 这三个在考试中通常是等价的，统称为 **保存し終了**。

正确答案

- `:e newfile`
- `:e! newfile`

考试小秘籍

在 LPI 考试中，看到 **“全て選択”** 且涉及文件操作时，一定要同时考虑 **“正常情况”** 和 **“强制执行 (!)”** 这两种可能。

你可以通过命令结尾的符号来快速判断它的“脾气”：

1. 没有符号（如 `:q`）：**温和派**。如果你有事没做完（没保存），它会温柔地提醒你“警告が出る”。
2. 感叹号 `!`（如 `:q!`）：**霸道派**。代表“我就要这么干！”它可以做到 **保存せず**（不保存）或者 **強制的に**（强制执行）。
3. 字母含义：
 - W = 写 (Write)

- **Q** = 走 (**Q**uit)
- **E** = 换 (**E**dit / **E**xchange)

コマンド	説明
h	1文字左へ移動
l	1文字右へ移動
k	1文字上へ移動
j	1文字下へ移動
0	行頭へ移動
^	行の先頭の文字へ移動
\$	行末へ移動
H	画面の最上行へ移動
L	画面の最下行へ移動
gg	ファイルの先頭へ移動
G	ファイルの最終行へ移動
nG	ファイルのn行目へ移動
:n	ファイルのn行目へ移動
Ctrl+b	1画面分、上方を表示
Ctrl+f	1画面分、下方を表示

 核心知识点总结：vi 移动命令记忆法

在 **コマンドモード**（命令模式）下，vi 的移动命令遵循从“字符”到“整行”再到“全局”的逻辑。

1. 基础方向移动（最常考的四个方向）

这四个键位于键盘中心，是 vi 操作的基础。

命令	考试关键词 (日语)	记忆口诀
h	1文字左へ移動	h 在最左边。
l	1文字右へ移動	l 在最右边。
k	1文字上へ移動	k 像个梯子往上爬。
j	1文字下へ移動	j 像个鱼钩往下钩。

1.2 行内定位移动（精准打击）

考试常考“行头”和“行末”的区别。

命令	考试关键词 (日语)	记忆逻辑
0 (零)	行頭 へ移動	数字 0 是绝对坐标的起点。
^	行の 先頭の文字 へ移動	排除空格，跳到第一个有字的地方。
\$	行末 へ移動	放在句子末尾的美元符号（收尾）。

1.3 文件全局移动（大跨度跳跃）

关键词通常包含「最初」「最終」「n行目」。

命令	考试关键词 (日语)	记忆口诀
gg	ファイルの 先頭 へ移動	go go! 回到最初。
G	ファイルの 最終行 へ移動	大写的 G 更有力量，直接跳到底。
nG / :n	ファイルの n行目	指定数字（n）跳过去。
H	画面の 最上行	High（最高处）。
L	画面の 最下行	Low（最低处）。

1.4 翻页操作

命令	考试关键词 (日语)	记忆逻辑
Ctrl+b	1画面分、上方	back（后退/往上）。
Ctrl+f	1画面分、下方	forward（前进/往下）。

🔍 考试关键词避坑指南

1. 区分 0 和 ^：
 - 如果题目关键词是「行頭」\$\\rightarrow\$ 选 0。
 - 如果关键词是「先頭の文字」（第一个字符）\$\\rightarrow\$ 选 ^。
2. 区分 gg 和 H：
 - gg 是跳到「ファイルの先頭」（整个文件的第一行）。
 - H 只是跳到「画面の最上行」（你现在眼睛能看到的这一屏的最上方）。
3. 记住 j 和 k 的上下方向：
4. 考试有时会出图示题，记住 j 是向下（小写字母 j 有个尾巴垂在下面）。

🧠 小学生级记忆法：身体部位记忆法

想象你的键盘就是你的身体：

- `h` / `l`：你的左右手。
- `k` / `j`：你的头和脚。
- `gg`：你的头顶（文件最上面）。
- `G`：你的脚底板（文件最后面）。
- `$`：你的裤兜（放在后面的钱）。

コマンド	説明
set	全てのシェル変数と環境変数を表示
declare	全てのシェル変数と環境変数を表示
env	全ての環境変数を表示
printenv	指定の、または全ての環境変数を表示

💡 核心知识点总结：变量显示命令对比

在 Linux 中，并不是所有变量都是平等的。有些只在当前 Shell 有效（シェル変数），有些则能遗传给子进程（環境変数）。

命令	考试关键词 (日语)	记忆口诀
set	全ての シェル変数 と 環境変数 を表示	Set = Super (最全)。管得最宽，全部都报。
declare	全ての シェル変数 と 環境変数 を表示	与 set 类似，通常作为 set 的等价选项出现。
env	全ての 環境変数 を表示	Environment（只看环境）。不带家室（Shell 变量）。
printenv	指定の、または 全ての 環境変数 を表示	Print（打印指定）。比 env 多了一个查看指定变量的功能。

🔍 考试关键词避坑指南

1. 关键词：全て (包含 Shell 变量)
 - 只要题目要求显示「シェル変数」，必须选 `set` 或 `declare`。
 - 坑点：`env` 绝对看不到纯粹的 Shell 变量。
2. 关键词：環境変数のみ (仅限环境变量)
 - 题目问显示「環境変数」时，`env` 和 `printenv` 是标准答案。虽然 `set` 也能看到，但考试通常倾向于考查专用的环境命令。
3. 关键词：指定の (指定的某个变量)

- 如果题目问如何显示「指定の」某个环境变量（例如只看 PATH），唯一专门提到的命令是 `printenv`（用法：`printenv PATH`）。

🧠 小学生级记忆法：班级大名单

想象变量是学校里的学生：

- `set` 老师：是 班主任。他手里有「全て」学生的名单，包括只在班里混的（Shell 变量）和要去参加全校活动的（环境变量）。
- `env` 老师：是 教导主任。他只关心「環境」（全校性质）的学生名单，班里的小事他不管。
- `printenv` 老师：是 点名老师。他不仅有全校名单，你告诉他一个名字，他还能专门把那个人「指定」叫出来。

关于 `env` 命令的执行逻辑，这是 Linux 认证考试中「シェル環境 (外壳环境)」分区的必考题目。

这一知识点主要考察 `env` 命令在不同参数下的双重身份：它既可以用来 一時的に環境変数を設定して実行（临时设置环境变量并执行命令），也可以用来 現在の環境変数を表示（显示当前环境变量）。

💡 知识点总结

`env` 命令的主要用途是在修改后的环境中运行程序。但是，如果没有指定任何命令或选项，它的默认行为是列出当前会话中所有的 環境変数 (环境变量)。

🔍 选项解析与详细说明

在考试中，关于执行 `env`（不加参数）的选项通常如下：

- ☒ 「設定されている環境変数の一覧を表示する」：正确答案。
 - 理由：当 `env` 后面没有跟随 `VARIABLE=VALUE` 或具体的 `COMMAND` 时，它会读取当前 shell 的环境并将其全部打印到标准输出。这与不加参数执行 `printenv` 命令的效果几乎完全一样。
- ☒ 「エイリアスの一覧を表示する」：错误。
 - 理由：查看 エイリアス (别名) 需要使用 `alias` 命令。`env` 只能看到环境变量，看不到别名。
- ☒ 「シェル変数の一覧を表示する」：错误。
 - 理由：シェル変数 (本地变量) 仅在当前 shell 有效，不会被子进程继承。要查看包含本地变量和环境变量的所有变量，通常使用 `set` 命令。`env` 只显示导出的环境变量。
- ☒ 「すべての環境変数を削除する」：错误。

- 理由：虽然 `env -i` 可以创建一个“清空”的环境来运行命令，但单纯执行 `env` 绝对不会删除任何变量。

 `env`、`set` 与 `printenv` 的排查对比表

コマンド (命令)	作用	考试关注点
<code>env</code>	显示环境变量 或 临时修改运行	不加参数 = 列出环境变量
<code>printenv</code>	显示环境变量	可以接特定变量名（如 <code>printenv PATH</code> ）
<code>set</code>	显示所有变量 (环境变量+本地变量)	范围最广，还包括 shell 函数

 小学生级记忆法：“环境探测器”

想象 `env` 是一个名为“环境 (Environment)”的探测器：

- 如果你给它任务（如 `env NAME=Gemini ./test.sh`）：
- 它会变成一个“临时变装间”。它让 `./test.sh` 以为自己叫 Gemini，但任务一结束，它就变回原样了。
- 如果你不给它任务（只输入 `env`）：
- 它会变成一个“照妖镜”。它会把此时此刻空气中漂浮的所有“环境因子”（环境变量）全都照出来给你看。

 正确答案

設定されている環境変数の一覧を表示する (显示已设置的环境变量列表)

 考试小秘籍

- 临时性：记住 `env` 修改变量是「一時的」的。它不会改变你当前 shell 的 `PATH` 或其他变量，只对它后面跟着的那个命令生效。
- 无参数行为：在 Linux 考试中，很多命令在不加参数时都有“列出当前状态”的默认行为（如 `ls` 列出目录，`mount` 列出挂载点，`env` 列出环境变量）。

这一知识点主要考察的是如何查看 **シェル変数 (Shell 变量)** 和 **環境変数 (环境变量)**，特别是区分哪些命令可以查看到“全部”变量。

💡 知识点总结

在 Linux 中，`set` 和 `declare` 是最全面的查看命令，它们可以列出 **全て (全部)** 的变量，包括 **シェル変数** 和 **環境変数**。

🔍 选项解析与详细说明

- `set`：正确答案。
 - **功能**：显示当前 Shell 中定义的所有变量（包括局部变量和导出的环境变量）以及定义的函数。
- `declare`：正确答案。
 - **功能**：在不带参数的情况下使用时，其效果与 `set` 类似，会列出当前 Shell 中 **全て** 的变量和函数。
- `env`：错误。
 - **理由**：它只能显示 **環境変数**（即那些被 export 导出的变量），无法看到普通的 **シェル変数**。
- `printenv`：错误。
 - **理由**：与 `env` 类似，它专门用于显示 **環境変数**，不支持显示纯粹的 **シェル変数**。
- `echo`：错误。
 - **理由**：`echo` 只能显示你 **指定** 的某一个变量的值（例如 `echo $PATH`），不能实现“一覽表示（列表显示）”。

📊 变量查看命令对比表

为了帮你快速通过考试，请记住这个包含关系：

命令	包含的内容 (日本語)	记忆口诀 (中文)
set	全て (シェル変数 + 環境変数)	Set 是 Super，管得最宽。
declare	全て (シェル変数 + 環境変数)	Declare 宣告 Da-quan (全)。
env	環境変数のみ	Env 只管 Environment (环境)。
printenv	環境変数のみ	只打印 Env。

🧠 小学生级记忆法：班级名单

想象你的电脑里有很多 **学生（变量）**：

- 1. **set 老师**：他是 **班主任**。他手里有一份「**全て**」的名单，无论这个学生是只在班里活动的（Shell 变量），还是要去参加全校活动的（环境变量），他全都知道。
- 2. **env 老师**：他是 **教导主任**。他只关心那些有资格去参加全校 **环境 (Environment)** 社交的学生，待在教室里的普通学生（Shell 变量）他不管。
- 3. **题目要求**：是要看“所有学生”，所以要找班主任 **set** 老师。

✅ 正确答案

- set
- declare

📝 考试小秘籍

考试中如果提到「**環境変数のみ**」，答案就是 **env** 和 **printenv**；如果提到「**全てのシェル変数**」，锁定 **set** 和 **declare**。

コマンド	説明
yy Y	カーソル位置の行をバッファにコピー
yw	カーソル位置の単語をバッファにコピー
dd	カーソル位置の行を削除し、バッファにコピー
dw	カーソル位置の単語を削除し、バッファにコピー
x	カーソル位置の文字を削除し、バッファにコピー
X	カーソル位置の左の文字を削除し、バッファにコピー
p	バッファの内容を挿入(文字や単語はカーソルの右、行はカーソルの下に挿入)
P	バッファの内容を挿入(文字や単語はカーソルの左、行はカーソルの上に挿入)
u	元に戻る(直前の編集をとりやめる)
.	繰り返す(直前の編集を再実行する)

💡 核心知识点总结：vi 编辑命令记忆法

在 vi 的 **コマンドモード（命令模式）** 下，大部分编辑指令都遵循“动作 + 范围”的逻辑。

1. 拷贝与删除（基础动作）

记住：在 vi 中，删除（d/x）其实就是剪切，内容会自动存入 **バッファ（缓冲区）**。

命令	考试关键词 (日语)	记忆逻辑
yy / Y	カーソル位置の 行 をコピー	yank（拔/取）。两次 y 代表整行。
yw	カーソル位置の 単語 をコピー	yank word。
dd	カーソル位置の 行 を削除	delete delete。两次 d 代表整行。
dw	カーソル位置の 単語 を削除	delete word。
x	カーソル位置の 文字 を削除	像在字符上打个叉。
X	カーソル位置の 左 の文字を削除	大写的 X 更有力，向后退着删（类似 Backspace）。

1.2 粘贴（大小写的乾坤大挪移）

粘贴命令 **p** 的位置取决于你复制的是“行”还是“字符”。

命令	考试关键词 (日语)	记忆口诀
p (小写)	下 に挿入 / 右 に挿入	小 p 往下钻（行下）或向右看。
P (大写)	上 に挿入 / 左 に挿入	大 P 往上顶（行上）或向左看。

1.3 撤销与重复（后悔药）

命令	考试关键词 (日语)	记忆口诀
u	元に戻す (直前の編集をとりやめる)	undo（撤销）。
.(点)	繰り返す (直前の編集を再実行)	点点点，继续干。

 考试关键词避坑指南

1. 区分 **x** 和 **X**：

- 关键词：「カーソル位置」 $\rightarrow$ 选 **x**。
- 关键词：「左の文字」 $\rightarrow$ 选 **X**。

2. 区分 **p** 和 **P** (针对“行”)：

- 关键词：「行の下」 $\rightarrow$ 选 **p**。
- 关键词：「行の上」 $\rightarrow$ 选 **P**。

3. 单字与单词：

- 关键词包含「単語」 $\rightarrow$ 命令必须带 **w** (yw, dw)。
- 关键词包含「行」 $\rightarrow$ 命令通常重复两次 (yy, dd)。

🧠 小学生级记忆法：三字经

- yy 复制，dd 删。
- p 在下，P 在上。
- u 是后悔，点 是重复。
- x 删自己，X 删左边。

💡 知识点总结：行頭 (行首) 移动的微妙差别

在 vi 的「コマンドモード (命令模式)」中，回到行首有两个常用的键：`0` (数字零) 和 `^` (脱字符)。

1. `0` (数字零)：绝对行首。不管开头有没有空格，直接跳到这一行的第 1 个位置。
2. `^` (ハット)：有效行首。跳到这一行第一个不是空格的字符上。

2. 📄 核心术语表 (用語集)

日语术语	中文含义	对应按键
行頭 (ぎょうとう)	行首	0 或 ^
文字の削除	删除字符	x (删除光标处)
次行へ移動	移至下行	j
空白を含む行頭	包含空格的行首	对应 0 的位置

2. 🏠 小学生记忆法：“滑梯”与“跳跳虎”

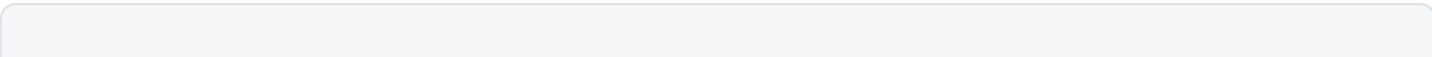
想象每一行文字前面都有几个空格（空白）。

- 按 `0` 键 (滑梯)：你从行末“咻”地一下滑到了最左边的墙角。如果墙角和文字之间有空位，你还得自己走过去。
- 按 `^` 键 (跳跳虎)：你像跳跳虎一样，直接跳过那些没用的空地，精准地踩在第一个有字的砖块上。

为什么这道题 `^` 更好？

观察图片里的文本：

Plaintext



```
1  # aA
2  # bB
```

每一行开头都有一个空格。如果按 `0`，光标会停在空格上，按 `x` 删掉的是空格。

而按 `^`，光标会直接踩在 `#` 上。接下来的动作就是：

1. `^`：踩在 `#` 上。
2. `x`：删掉 `#`。
3. `x`：删掉后面的空格。
4. `j`：跳到下一行。
5. `^`：踩在第二行的 `#` 上（注意：`j` 之后光标位置不固定，再按一次 `^` 最保险）。
6. `xx`：删掉第二行的 `#` 和空格。

3. ✂️ 实战演习：答案修正

题目要求：从 1 行目行末的 `A` 开始，删除全行的 `#` 和空格。

- 你的答案：`^xxj^xx`
 - 第一步：`^` 瞬间跳过前面的空格，踩死 `#`。
 - 第二步：`xx` 连发两枪，消灭 `#` 和它后面的空格。
 - 第三步：`j` 降落到第二层。
 - 第四步：`^` 再次找准第二层的 `#`。
 - 第五步：`xx` 完成收尾。

这个操作比 `0` 更智能，因为它能自动过滤掉行首可能存在的干扰空格！

🌟 考试快速查阅表

需求	按键	记忆口诀
跳到绝对最左边	<code>0</code>	从零开始
跳到第一个字	<code>^</code>	戴上帽子找文字
删除光标处	<code>x</code>	打个叉叉
删除光标前	<code>X</code> (大写)	向后开枪

这一知识点主要考察 **viエディタ (vi编辑器)** 在 **コマンドモード (命令模式)** 下，针对 **削除 (x)**、**移動 (k)** 以及 **アンドゥ (u)** 和 **リピート (.)** 命令的组合应用逻辑。

💡 知识点总结

在 vi 中，**u (undo)** 用于撤销上一次的**编辑 (修改) 操作**，而 **.(repeat)** 用于重复上一次的**编辑 (修改) 操作**。需要注意的是，单纯的**移動操作 (如 h, j, k, l)** 不被视为“编辑操作”，因此不会被 **u** 撤销，也不会被 **.** 重复。

🔍 选项解析与详细说明

我们来拆解题目中的指令序列：

3.1 关于「xku」的分析

- **指令分解：**
 - a. **x**：执行 **削除 (删除)** 操作，这是一个编辑动作。
 - b. **k**：执行 **上へ移動 (向上移动)** 操作，这只是光标移动，不改变内容。
 - c. **u**：执行 **元に戻す (撤销)**。
- **结果：**由于 **k** 不是编辑操作，**u** 会跳过 **k**，直接撤销在它之前的编辑动作 **x**。
- **对应选项：**「xku」と入力した場合、削除の操作が取り消される（正确）。

3.2 关于「xk.」的分析

- **指令分解：**
 - a. **x**：执行 **削除 (删除)** 操作，这是上一次的编辑动作。
 - b. **k**：执行 **上へ移動 (向上移动)** 操作，不被 **.** 记录。
 - c. **.**：执行 **繰り返す (重复)** 上一次的编辑动作。
 - **结果：****.** 会重复之前的 **x** 操作。也就是说，在当前光标位置再次删除一个字符，其效果等同于再次输入了 **x**。
 - **对应选项：**「xk.」と入力した場合、再度「x」を入力した結果と同様になる（正确）。
-

📊 vi 编辑与移动逻辑对照表

为了帮你像小学生一样记牢，请记住这个关键规则：

操作类型	具体例子	是否受 u 或 . 影响?	记忆口诀
編集操作	x, dd, yy, p	是	动了真格（改了内容），就能撤回或重做。
移動操作	h, j, k, l, o, \$	否	只是看看（跑个位），u 和 . 都不理它。

小学生级记忆法：隐身的光标

想象你在玩一个盖章游戏：

- x** (盖章/涂改)：你在纸上涂了一个黑点。
- k** (走路)：你拿着笔往上走了一步，但没动笔。
- u** (时光倒流) / **.** (复印机)：
 - 因为刚才“走路”没留下痕迹，所以时光倒流 **u** 会直接把那个“黑点”擦掉。
 - 复印机 **.** 也会无视你刚才走的路，直接在现在的位置再帮你涂一个同样的“黑点”。

✓ 正确答案

- 「xku」と入力した場合、削除の操作が取り消される
- 「xk.」と入力した場合、再度「x」を入力した結果と同様になる

✎ 考试小秘籍

LPI 考试非常喜欢出这种「编辑 + 移动 + 撤销/重复」的组合题。你只需要记住一句话：光标移动是透明的，**u** 和 **.** 永远只盯着你上一次动过的那个“文字”不放。

这一知识点主要考察在进行 **アンマウント（卸载挂载点）** 操作时，遇到「使用中（Device is busy）」错误后的正确处理流程。

💡 知识点总结

当文件系统无法卸载时，通常是因为有进程正在访问该目录下的文件，或者有用户的 **カレントディレクトリ（当前工作目录）** 位于该挂载点内。最稳妥、最符合管理规范的处理方式是结束相关的读写进程或让用户退出该目录。

🔍 选项解析与详细说明

- 「関連ファイルを操作中のユーザに、操作を終了してもらう」：正确答案。

- **理由**：这是最安全且标准的操作。作为管理员，应首先确认哪些进程或用户正在使用该设备。可以使用 `fuser -m` 或 `lsof` 命令来查找使用中的用户和进程。
- 「**umountコマンドにより強制的にアンマウントする**」：错误。
 - **理由**：虽然 `umount -f` 或 `umount -l` 可以强制执行，但这可能导致 **データ破損（数据损坏）** 或文件系统不一致。在生产环境下，除非万不得已，否则不应首先尝试。
- 「**管理者権限に変更して安全にアンマウントする**」：错误。
 - **理由**：卸载操作通常本身就需要 **管理者権限 (root)**。如果你已经在用 root 权限执行但依然报错 “Device is busy”，说明问题在于文件被占用，切换权限无法解决占用问题。
- 「**/proc/self/mounts**」と「**/proc/mounts**」を**書き換える**」：错误。
 - **理由**：`/proc` 下的文件是 **カーネル（内核）** 信息的实时映射，不能通过手动改写这些虚拟文件来执行卸载动作。

处理“设备忙”问题的排查表

为了帮你像小学生一样记牢，请记住这套“三步走”方案：

步骤 (日本語)	操作 (中文解释)	常用命令
1. 調査	确认谁在占用设备。	<code>fuser -mu [挂载点]</code>
2. 通告/終了	请用户退出或结束进程。	<code>kill</code> 或直接沟通
3. 実行	确认没人在用后卸载。	<code>umount [挂载点]</code>

小学生级记忆法：别拽桌布

想象你在玩“抽桌布”的游戏（卸载挂载点）：

1. **有人在吃饭**：如果桌子上还有人正在吃饭（用户正在操作文件），你猛地一拽桌布，碗筷全碎了（数据损坏）。
2. **正确做法**：你要温和地告诉吃饭的人：“**我要撤桌子了，请大家吃完离开（操作終了）**”。等桌子空了，你才能轻而易举地把桌布抽走。

正确答案

関連ファイルを操作中のユーザに、操作を終了してもらう

考试小秘籍

在 LPI 考试中，系统管理题目往往倾向于考察“**最安全、最正规**”的方法。虽然“强制执行”在技术上可行，但在选项中，“先解决根源（进程占用）”通常才是标准答案。

这份截图完整展示了在 Linux 中处理「**デバイスが使用中 (Device is busy)**」导致无法卸载文件系统的标准流程。

💡 知识点总结

当一个目录被某个进程 **占用**（比如有程序正在读取该目录下的文件）时，直接执行 `umount` 会失败。必须先「**特定（识别）**」并「**終了（终止）**」占用该目录的进程，才能成功卸载。

🔍 流程实战拆解（按图片步骤）

1. 准备环境：挂载与模拟占用

- **操作：**使用 `mount` 将设备挂载到 `/mnt/testpoint`。
- **模拟占用：**执行 `tail -f /mnt/testpoint/file1.txt &`。
 - **关键点：**`tail -f` 会一直保持文件打开状态，`&` 让它在后台运行。此时，该文件系统正处于「**使用中**」状态。

```
マウント
# mount /dev/sdb1 /mnt/testpoint

マウントしたファイルシステムのファイルにアクセスするコマンドを実行
(バックグラウンドで継続実行)
# tail -f /mnt/testpoint/file1.txt &
[1] 3046

アンマウントを試みると、ファイル操作中のために失敗する
# umount /mnt/testpoint
umount: /mnt/testpoint: target is busy.
(In some cases useful info about processes that use
the device is found by lsof(8) or fuser(1))

該当のファイルシステムを使用中のプロセスを確認
# fuser -m /mnt/testpoint
/mnt/testpoint:      3046
ファイル操作を終了
# kill 3046
# jobs
[1]+  Terminated                  tail -f /mnt/testpoint/file1.txt &

アンマウントすると成功し、エラーメッセージが表示されない
# umount /mnt/testpoint
```

3.2 报错：卸载失败

- **操作：**尝试执行 `umount /mnt/testpoint`。
- **错误信息：**`target is busy`。
 - **提示：**系统友好地提示你可以使用 `lsof` 或 `fuser` 来查找原因。

3.3 核心步骤：查找占用者 (fuser)

- **命令：**`fuser -m /mnt/testpoint`。
- **结果：**显示 PID 为 `3046`。
 - **含义：**这明确告诉你，进程号 3046 正拽着这个目录不撒手。

3.4 解决问题：清除占用并卸载

- **命令：**`kill 3046`。

- **动作**：强制结束（终止）那个占用的进程。
- **最终卸载**：再次执行 `umount /mnt/testpoint`，没有任何错误提示，表示 **成功卸载**。

 核心工具对照表

工具 (日本語)	作用 (中文解释)	记忆口诀
mount	デバイスを接続する	把设备“插”进系统。
umount	デバイスを切り離す	把设备“拔”出来。
fuser -m	使用中のプロセスを確認	找找谁在“抓”着设备不放。
kill	プロセスを終了させる	撒不开手？直接把它“关掉”。

 小学生级记忆法：谁在踩我的桌布？

想象你在收拾餐桌（卸载目录）：

1. **抽不动**：你想把桌布（文件系统）抽走，结果系统报错说“太重了，有人在用 (Target is busy)”。
2. **找凶手**：你低头一看（执行 `fuser`），发现有个小孩（进程 3046）正踩在桌布上。
3. **请走他**：你让他离开或者把他抱走（执行 `kill`）。
4. **收桌子**：小孩走了，你轻轻一拉（执行 `umount`），桌布就收好了。

 考试重点提醒

在考试中，如果题目问到「アンマウントできない原因を調べるコマンド」（调查无法卸载原因的命令），请务必在选项中寻找 `fuser` 或 `lsof`。

メタキャラクタ	説明
*	0文字以上の文字列
?	任意の1文字
[]	[]内のいずれか1文字 [abc] a,b,cのどれか1文字 [a-z] 小文字のアルファベット1文字 [!abc] a,b,c以外の1文字
\$	変数
'	「'」で囲まれた部分を文字列と見なす (メタキャラクタの意味を打ち消す)
“ ”	「”」で囲まれた部分を文字列と見なす (メタキャラクタの意味を打ち消す。 ただし「\$」「`」「¥」「”」は除く)
`	「`」で囲まれた文字列(変数の場合は 格納されている値)をコマンドと見なし、 その実行結果を文字列として返す
¥	次の文字(メタキャラクタ)の意味を 打ち消して通常の文字として扱う (エスケープする)
ディレクトリの特殊記号	
~	ホームディレクトリ
.	カレントディレクトリ
..	カレントディレクトリの1つ上の階層

这份截图 汇总了 Linux 命令行中极其重要的 **メタキャラクタ**（元字符/通配符）和 **ディレクトリの特殊記号**（目录特殊符号）。它们是编写命令和脚本的“缩写密码”。

💡 核心知识点总结：元字符与目录符号记忆表

我将表格内容按功能逻辑进行了拆解，重点标注了考试中最常出现的 **解释关键词**。

3.5 通配符：用于匹配文件名（文件名展开）

元字符	考试关键词 (日语)	记忆口诀
*	0文字以上 の文字列	贪吃星。管它多少个字，全部匹配。
?	任意の1文字	问号代表“这一个字是谁？”。
[]	内の いずれか1文字	选择框。里面写啥就挑啥。
[a-z]	小文字 のアルファベット1文字	范围连连看。
[!abc]	a,b,c 以外の1文字	!代表“不是”，排除掉它们。

3.2 引用符号：决定“谁说了算”

这是 LPI 考试的重灾区，请务必区分三种引号的区别。

符号	考试关键词 (日语)	记忆逻辑
' '(单引号)	メタキャラクタの 意味を打ち消す	强力封印。里面所有的特殊符号都变成普通的文字。
" "(双引号)	\$ ` \ は 除く	部分封印。保留变量调用 (\$) 等核心功能，其余封印。
` (反引号)	実行結果 を文字列として返す	命令替换。先运行里面的命令，把结果拿出来用。
\ (反斜杠)	次の文字を エスケープ する	单体封印。只让紧跟在后面的那一个符号失效。

3.3 目录与变量特殊符号

符号	考试关键词 (日语)	记忆逻辑
\$	変数	钱符号代表“取值”。
~	ホーム ディレクトリ	波浪号代表“家 (Home)”。
.	カレント ディレクトリ	一个点代表“我就在这里”。
..	1つ上の階層	两个点代表“往上跳一层”。

🔍 考试关键词避坑指南

1. 区分 * 和 ? :

- 关键词：「0文字以上」 \$\rightarrow\$ 选 * 。
- 关键词：「1文字」 \$\rightarrow\$ 选 ? 。

2. 区分 [abc] 和 [!abc] :

- 关键词：「いずれか」（其中之一） \$\rightarrow\$ 选 [abc] 。
- 关键词：「以外」（除了...） \$\rightarrow\$ 选 [!abc] 。

3. 单双引号的大招:

- 如果题目说要让所有的元字符都失去意义（打ち消す），最彻底的就是 ' ' (单引号)。

🧠 小学生级记忆法：三口之家

- ~ (波浪): 是你的家，舒服得像水波纹。
- . (一点): 是你站的位置，就在脚下。
- .. (两点): 是两级台阶，让你爬到上一层去。
- ' ' (单引号): 是一把铁锁，锁住所有元字符，不让它们变魔术。

这一知识点主要考察的是 **ワイルドカード（通配符）** 中 **[]** 符号的匹配规则，特别是它如何处理 **いずれか1文字（其中任意一个字符）**。

💡 知识点总结

在通配符规则中，**[abc]** 意味着匹配括号内列出的 **a、b 或 c 中的任意一个字符**。它并不是要排除这些字符，也不是要匹配整个单词 "abc"。

🔍 选项解析与详细说明

```
$ ls
A.txt a-z.txt abc.txt b.txt x.txt
$ ls [abc].txt
```

根据上图当前目录下有以下文件：

- A.txt
- a-z.txt
- abc.txt
- b.txt
- x.txt

当你执行命令 `ls [abc].txt` 时：

- **b.txt**：正确答案。
 - **理由**：文件名以 `b` 开头，且 `b` 包含在 `[abc]` 范围内，后面紧跟 `.txt`。
- **x.txt**：错误。
 - **理由**：`x` 不在 `[abc]`（a、b、c）这三个选项中。如果你想要“不是 abc”的效果，应该使用 `[!abc].txt`。
- **abc.txt**：错误。
 - **理由**：`[abc]` 只能代表 **1 个字符**。`abc.txt` 的开头有三个字符 "abc"，不符合匹配规则。
- **A.txt**：错误。
 - **理由**：通配符通常区分大小写。`A` 不等于 `a`。
- **a-z.txt**：错误。
 - **理由**：这个文件名的开头是 "a-z"，同样超过了 1 个字符的限制。

📊 通配符匹配逻辑对比表

为了帮你纠正“排除”的误解，请看这个对比：

符号 (日本語)	匹配规则 (中文)	匹配示例 (基于你的文件)
<code>[abc].txt</code>	必须是 a 或 b 或 c 其中之一	b.txt
<code>[!abc].txt</code>	除了 a、b、c 以外的任意一个字	x.txt
<code>abc.txt</code>	必须精确匹配 "abc" 这三个字	abc.txt

🧠 小学生级记忆法：三选一的小篮子

想象你在玩一个抽奖游戏，规则写在 `[]` 盒子里：

1. `[abc]`：这是一个“中奖名单”。篮子里放着三张小纸条：`a`、`b`、`c`。
2. 验证文件名：
 - 看到 `b.txt`：名字开头是 `b`。老师检查名单，发现 `b` 在篮子里，**中奖了!**
 - 看到 `x.txt`：名字开头是 `x`。老师检查名单，发现没有 `x`，**没中奖**。
3. 误区提醒：只有当篮子里写着 `!`（不）的时候，即 `[!abc]`，才代表“只要不是名单上的都算中奖”。

✅ 正确答案

`b.txt`

📝 考试小秘籍

LPI 考试中，`[]` 永远代表「**いずれか1文字**」（其中任意一个字符）。

- 如果你看到 `[a-c]`，这和 `[abc]` 是一样的，代表一个范围。
- 如果你看到 `[!...]` 或 `[^...]`，这才是你想要的“排除”。

以下の内容のテキストファイルFileがある。

cat File | sed s/r/O/i を実行した結果はどれか。

```
X r r
R r r
```

这一知识点主要考查 `sed` 命令的 **文字列置換（字符串替换）** 功能，特别是针对 **フラグ（标志位）** 的应用。

💡 核心知识点总结：sed 置换逻辑

`sed` 的基本语法为 `s/置換対象/置換後/フラグ`。在本题中，命令 `s/r/O/i` 的关键在于最后的 `i` 标志。

- `s`：代表 **Substitute（置换）**。
- `/r/`：查找所有的字母 `r`。

- `/O/`：将其替换为大写字母 **O**。
- `i`：代表 **Ignore case**（忽略大小写）。这意味着无论是小写 `r` 还是大写 `R` 都会被匹配并替换。

选项解析与详细过程

根据你提供的原始文件 **File** 内容：

Plaintext

代码块

```
1  X r r
2  R r r
```

执行 `sed s/r/O/i` 后的变化如下：

1. 第一行： `X r r`
 - 第一个 `r` 被替换为 `O` $\rightarrow$ `X O r`
 - **注意：**由于命令中没有 `g` (global) 标志，`sed` 默认只替换每一行中 **第一个** 匹配到的字符。
 - 结果： `X O r`。
2. 第二行： `R r r`
 - 第一个字符是 `R`。
 - 因为使用了 `i` 标志，大写 `R` 被视为匹配项。
 - `R` 被替换为 `O` $\rightarrow$ `O r r`
 - 同样，因为没有 `g`，该行后续的 `r` 不动。
 - 结果： `O r r`。

sed 标志位对比表

为了帮你像小学生一样记牢，请记住这两个决定替换“范围”和“深度”的开关：

标志 (日本語)	功能 (中文解释)	记忆口诀
<code>i</code>	大文字・小文字を 区別しない	ignore，大小写一家亲。
<code>g</code>	行内の 全て を置換	global，全行一扫光。
无标志	最初の1つだけ置換	只动排头兵。

🧠 小学生级记忆法：点名游戏

想象你在给班级里的同学换座位：

1. `s/r/o/`：老师说：“叫 `r` 的同学，把衣服换成 `o` 颜色的。”
2. `i` 标志：老师补充：“不管是大写的 `R` 还是小写的 `r`，只要叫这个名字的都算。”
3. 没喊 `g`：老师没说“全部”，所以 **每一排（每一行）只有第一个** 听到名字的同学会去换衣服。
4. 结果：
 - 第一排的 `r` 换了，后面的 `r` 没动。
 - 第二排开头的 `R` 换了，后面的 `r` 没动。

✅ 正确答案

对应的图片结果是：

```
X o r
O r r
```

📝 考试小秘籍

LPI 考试最喜欢考 `g` 和 `i` 的组合。

- 如果题目是 `s/r/o/g` $\rightarrow$ 答案会是所有 `r` 都变 `o`，但 `R` 不变。
- 如果题目是 `s/r/o/gi` $\rightarrow$ 答案会是全屏所有的 `r` 和 `R` 全都变成 `o`。

这一知识点主要考查 `sed` 命令中「`&`」记号的特殊用法，以及如何通过 **フラグ（标志位）** 实现全行置换。

💡 核心知识点总结：sed 的 `&` 符号与 `g` 标志

在 `sed` 的置换字符串中，`&` 是一个特殊的变量，它代表「**マッチした文字列全体**」（匹配到的整个字符串内容）。而为了让置换作用于每一行中 **所有** 匹配到的项，必须在命令末尾添加 `g` (**global**) 标志。

🔍 题目解析与填空

根据你上传的图片信息：

- File 的内容： Book book BOOK
- 输出结果： A Book. A book. A BOOK.

以下のコマンドの下線部に何をいれるべきか。
\$ cat File | sed 's/Book/A &./__'

Fileの内容：
Book book BOOK

出力結果：
A Book. A book. A BOOK.

- ☒ gi
- ☐ y
- ☐ y,d
- ☐ g

3.4 理解 sed 's/Book/A &./' の逻辑

- 查找目标： Book
- 置换内容： A &.
 - 这里 & 会自动替换为它实际匹配到的内容。
 - 例如，匹配到 Book 时，结果变为 A Book.。
 - 由于 sed 的查找默认是区分大小写的，为了匹配 Book、book 和 BOOK，通常还需要配合 i（忽略大小写）标志。

3.2 为什么下划线处要填 gi？

- g (global)：观察输出结果，原本一行里的 三个单词全都被修改了。如果不加 g，sed 只会修改第一个单词。
- i (ignore case)：原始文件包含不同大小写的 "book"。为了用一个 Book 模式匹配到所有变体，必须使用 i 标志。

📊 sed 特殊符号与标志对照表

为了帮你像小学生一样记牢，请记住这两个“超级助手”：

符号 (日本語)	功能 (中文解释)	记忆口诀
&	マッチした文字列	原封不动搬过来。
g	行内の全て を置換	global，全行通杀。
i	大小文字を区別しない	ignore，大小写无所谓。

🧠 小学生级记忆法：复读机与全场扫描

- 1. **&** 是“复读机”：
 - 老师说：“在每个名字前面加个‘A’，后面加个‘点’。”
 - 你不用重新写名字，直接用 **&** 让系统自己把抓到的名字“复读”一遍放在中间。
- 2. **gi** 是“全场通报 + 别管大小写”：
 - i**：不管你是大写的书还是小写的书，都给我站出来。
 - g**：每一排里所有的书都要改，不能只改第一个。

✅ 正确答案

gi

📝 考试小秘籍

在 LPI 考试中，只要看到输出结果里 **一行之中** 有多个地方被改动了，百分之百要选带 **g** 的选项。如果同时涉及到了大小写混匹配，那就选 **gi**。

这是一道非常典型的 **sed 命令** 考试题，考查的是对「置換 (ちかん)」作用范围的理解。

💡 **核心知识点：** **sed** 命令在没有 **g** 选项时，默认只替换每行中出现的“第一个”匹配项。

4. 核心术语表 (用語集)

日語术语	中文含义	考试关键逻辑
置換 (ちかん)	替换	将 A 变成 B 的操作
行単位 (ぎょうたんい)	行单位	sed 是按行处理文本的
フラグ (Flag)	标志/后缀	放在命令末尾，改变处为 (如 g)
グローバル置換	全局替换	匹配行内所有的目标，选项 g

以下の内容のテキストファイルFileがある。
cat File | sed s/Black/White/ を実行した結果はどれか。

Black Black Black
Black Black

- ☒ White Black Black
White Black
- ☐ White White White
Black Black
- ☐ White White White
White White
- ☐ White Black Black
Black Black

2. 命令深入剖析 (コマンドの解説)

执行的命令是: `sed s/Black/White/`

- **s (substitute):** 表示进行 置換。
- **/Black/**: 查找的目标 (検索パターン)。
- **/White/**: 替换后的内容。
- **关键点:** 在最后一个 **/** 后面没有任何字符 (没有 **g**)。

🔧 逻辑拆解:

1. 第一行: `Black Black Black`

- `sed` 扫描这一行, 发现第一个 `Black`, 将其替换为 `White`。
- **重点:** 因为没有 **g** (Global) 标志, 它完成第一次替换后就“收工”了, 后面的两个 `Black` 保持不变。
- 结果: `White Black Black`

2. 第二行: `Black Black`

- 同理, `sed` 进入新的一行, 重新开始寻找第一个匹配项。
- 它把这行的第一个 `Black` 替换为 `White`, 第二个 `Black` 不动。
- 结果: `White Black`

3. 选项对比 (選択肢の確認)

- **正确选项 (第一个):** `White Black Black`
- `White Black`
- 符合我们刚才的逻辑: **每一行只换第一个**。
- **错误干扰项分析:**
 - 如果是 `sed s/Black/White/g`: 结果会是全部变成 `White` (第三个选项)。
 - 如果是只换第一行的第一个: 那是对 `sed` 行号的限制, 不符合默认逻辑。

★ 记忆秘籍 (覚え方)

在 Linux 认证考试中, 看到 `sed` 的 **s** 命令:

- 没有 **g** = けち (吝啬): 每一行只肯干一次活, 换完第一个就下班。
- 带了 **g** = グローバル (Global): 这一行里只要看到的, 全部换掉。

下次考试如果你看到要求“把文件中所有的 A 换成 B”，记得一定要在选项里找那个带 `g` 的！

🎨 知识点总结：爱偷懒的粉刷匠 (sed s///)

在 Linux 的世界里，`sed` 就是一个拿着刷子的粉刷匠，他的工作是把「黑色 (Black)」的墙面刷成「白色 (White)」。

4. 故事背景 (設定)

有一座两层的小楼：

- **第一层 (1行目)**：有 3 块黑砖 `Black Black Black`
- **第二层 (2行目)**：有 2 块黑砖 `Black Black`

2. 粉刷匠的工作习惯 (動作の論理)

这个粉刷匠非常“按部就班”且“爱偷懒”：

- **按层工作 (行単位)**：他先去第一层，干完活再去第二层。
- **见好就收 (デフォルトの動作)**：每到一层楼，他只要刷完**第一块**看见的黑砖，就立刻坐下喝茶，不再管剩下的砖了。

🖌️ 现场施工图解 (シミュレーション)

我们可以看看他是怎么干活的：

第一步：来到第一层 (1行目)

- 墙上是： `Black Black Black`
- 粉刷匠刷了第 1 块：变成 `White`。
- 他觉得任务完成了，剩下的 2 块 `Black` 他**看都不看**。
- **结果**： `White Black Black`

第二步：来到第二层 (2行目)

- 墙上是： `Black Black`
- 他重新开始工作，刷了这一层的第 1 块：变成 `White`。
- 剩下的 1 块 `Black` 他又**不管了**。
- **结果**： `White Black`

🏆 考试必杀技 (試験のコツ)

在考试中，你只要记住两句话，就能秒杀这种题：

- 1. 看到 `s/A/B/`（后面空空的）：他是偷懒粉刷匠，每行只换第一个。
- 2. 看到 `s/A/B/g`（后面有个 `g`）：这个 `g` 代表「Global (全部)」。这时候他就像打了鸡血，会把每一行所有的黑砖都刷成白色。

考试直感：题目给的命令是 `sed s/Black/White/`，后面没有 `g`。所以你盯着选项看，只要发现哪一行的第二个 `Black` 也变色了，那一定是错的！

这道题考察的是 **viエディタ (vi 编辑器)** 的基本操作。在 Linux 认证考试中，vi 的三种模式切换是绝对的必考点。

💡 知识点总结：モードの切り替え (模式切换)

vi 编辑器通过不同的「モード」来区分“写字”和“发号施令”。从「入力モード (输入模式)」回到「コマンドモード (命令模式)」，必须使用 **Esc** 键。

1. 核心术语表 (用語集)

日本語术语	中文含义	考试关键逻辑
コマンドモード	命令模式	默认模式，用于移动光标、删除、复制 (按 Esc 进入)
入力モード	输入模式	编写文字内容的模式 (按 i, a, o 等进入)
末尾行モード	底行模式	输入 : 后进行保存 (:w) 或退出 (:q)
エスケープキー	Esc 键	无论在哪，按它就能退回命令模式的“万能键”

2. 🏠 小学生记忆法：“躲猫猫”与“安全屋”

我们把 vi 编辑器想象成一个正在玩“躲猫猫”的小朋友：

- **输入模式 (打字中)**：就像你外面跑得满头大汗（一直在辛苦打字）。
- **命令模式 (待机中)**：就像你的「安全屋」。只有回到安全屋，你才能休息，或者决定下一步是去吃饭（保存）还是去睡觉（退出）。

怎么回安全屋呢？

你要大喊一声 “**Escape! (逃跑!)**”，也就是按一下键盘左上角的 **Esc** 键。

记法： **Esc = E-s-cape** (逃跑)。

只要想停止打字、想发指令，就赶紧“逃离”输入状态，按 **Esc** 回到安全屋。

3. 実戦演習 (题目解析)

问题：viエディタで入力モードからコマンドモードへ変更するキーは次のうちどれか。

-  **Esc**：正确答案。它是所有模式回归 **コマンドモード** 的唯一通道。
-  **Tab**：在输入模式下，它只是帮你跳格（缩进），不会切换模式。
-  **Ctrl / Shift / Alt**：这些是功能组合键，单独按它们无法实现模式转换。

4. 考试避坑指南

考试时，如果题目问「保存せずに終了する (不保存直接退出)」的命令是什么？

1. 先按 **Esc** (回到命令模式)。
2. 输入 `:q!` (进入底行模式并强制退出)。

重点：所有的冒号命令（如 `:wq`）都必须先确保你在 **コマンドモード** 下才能输入。所以如果不确定，先狂按 **Esc** 准没错！

这道题目考查的是 Linux 中最常用的「パイプライン (管道符)」组合操作，涉及排序、去重、列提取以及重定向。

知识点总结：テキスト処理のコンボ (文本处理组合拳)

要删除重复行，必须先用 `sort` 排序，再用 `uniq` 去重；提取列使用 `cut`；最后用 `>` 将结果保存到新文件。

1. 核心术语表 (用語集)

日本語术语	中文含义	考试关键逻辑
同一の行を削除	删除重复行	核心组合：sort → uniq
列を取り出す	提取列	使用 cut 命令，配合 -f (field) 参数
パイプ ()	管道	把前一个命令的“出口”接到下一个的“入口”
リダイレクト (>)	重定向	把最终结果从屏幕改送到文件

2. 小学生记忆法：“整理乱糟糟的卡片”

想象你手里有一叠乱七八糟的卡片，上面写着名字和编号，你想整理出一份干净的名单：

1. **第一步：排排队 (`sort`)**

- 乱放的话你根本不知道谁重复了。所以要先按字母顺序**排好队**。

2. 第二步：扔掉多余的 (`uniq`)

- 队伍排好后，你会发现相同的卡片都挨在一起了。这时候，把**长得一模一样的**卡片只留一张，剩下的扔进垃圾桶。

3. 第三步：剪下你想要的 (`cut`)

- 现在每张卡片都是唯一的了，但你只需要上面的第 1 个字和第 4 个字。拿剪刀把这两部分**剪下来**。

4. 第四步：放进新盒子 (`>`)

- 最后，把这些剪下来的纸片全部装进一个叫 `NewFile` 的**新盒子**里。

3. 実戦演習 (题目解析)

问题分析：

- **删除重复行**：必须是 `sort File | uniq`。注意：`uniq` 只能删除**相邻**的重复行，所以如果不先 `sort`，散落在各处的重复行是删不掉的。
- **提取 1 和 4 列**：命令是 `cut -f 1,4`。
- **存入 NewFile**：使用 `> NewFile`（霸道总裁覆盖式，因为文件本身不存在）。

选项对比：

-  `uniq File | sort ...`：顺序反了，不能保证完全去重。
-  `sort File | uniq | cut -d ',' -f 1,4 > NewFile`：完美符合逻辑。
-  `... < NewFile`：`<` 是从文件读入，我们要的是写入。
-  `-f 0,3`：Linux 的列计数通常从 **1** 开始，没有第 0 列。

正确答案：第二个选项 (`sort File | uniq | cut -d ',' -f 1,4 > NewFile`)

4. 考试避坑指南

- **顺序陷阱**：看到「重複行を削除」，脑子里第一反应必须是 `sort` 在前，`uniq` 在后。
 - **方向陷阱**：`>` 指向文件（存入），`<` 指向命令（读入）。考试时看箭头尖端指向哪里，指向文件就是保存。
 - **列号陷阱**：`cut` 的第一列是 `1`，不是 `0`。
-

sortコマンド

sort [オプション] [ファイル名]

※ 「-r」 オプションを指定しない限り、ファイルの内容は昇順にソートされます。

既然你正在攻克 `sort` (ソート) 命令，这在 Linux 认证中是处理文本数据的“大管家”。我们结合你上传的图片，用“日中结合”和“小学生记忆法”来彻底搞定它。

 知识点总结： `sort` 命令是按行整理数据的“排队官”

`sort` 默认按照**字符顺序**（ASCII 码）对文本进行升序排列。如果你想改变排队规则（比如按数字大小、倒序、或者忽略大小写），就必须使用 **オプション** (选项)。

1. 核心术语表 (用語集)

根据你上传的图片，这些是考试最高频的选项：

日語术语	中文含义	记忆关键点
降順でソート	降序排列	使用 -r (从大到小，Z 到 A)
数値としてソート	按数值大小排列	使用 -n (识别 10 比 2 大)
区切り文字	分隔符	使用 -t (如指定逗号或冒号)
フィールドを指定	指定字段/列	使用 -k (指定按哪一列排)
大文字小文字を区別しない	不区分大小写	使用 -f
行頭の空白を無視	忽略行首空格	使用 -b

オプション	説明
-b	行頭の空白を無視
-f	大文字、小文字を区別しない
-r	降順でソート
-t 区切り文字	指定した文字を区切り文字としてフィールドを認識
-n	数字を文字でなく数値としてソート
-k フィールド	ソート対象とするフィールドを指定 (デフォルトは1つ目のフィールド)

2. 小学生记忆法：“森林学校的小动物排队”

想象你是森林学校的老师，要给一群小动物排队。

- `-r` (Reverse) —— “倒立排队”**
 - 平时是矮个子在前，Reverse 就是反过来，让大个子站到最前面去！
- `-n` (Number) —— “看胸前的号码牌”**
 - 如果小动物胸前贴着数字 `2` 和 `10`。不加 `-n`，老师只看第一个数字，会觉得 `10` 以 `1` 开头所以排在 `2` 前面（这叫字符序）。加上 `-N`，老师才会把它们当成真正的**数字**，让 `10` 排在 `2` 后面。
- `-f` (Fold) —— “不看帽子颜色”**

- 大写字母 `A` 戴红帽子，小写字母 `a` 戴绿帽子。加上 `-f`，老师就把颜色“折叠” (Fold) 起来，觉得它们都是一样的 `a`。
 - `-k` (Key) —— “看第几列横队”
 - 小动物排成了几路纵队。`-k 2` 就是告诉老师：“别看第一排，盯着第 2 排的身高来排队！”
 - `-t` (Tag) —— “中间的挡板”
 - 如果小动物之间用逗号隔开，`-t ','` 就像给老师一把刀，告诉他：“看到逗号就切开，那是一个个隔间。”
-

3. ✂️ 实战演习 (题目模拟)

问题：某文件中保存了学生姓名和成绩（用冒号 `:` 分隔），如下所示。你想按成绩（第二列）从高到低排列，正确的命令是？

Tanaka:80

Sato:95

Suzuki:70

- A. `sort -t ':' -k 2 File`
 - B. `sort -t ':' -k 2 -r -n File`
 - C. `sort -k 2 -n File`
-

4. 考试解题逻辑 (解法)

1. 找分隔符：看到冒号，必须有 `-t ':'`。（排除 C）
 2. 找排哪列：成绩在第二列，必须有 `-k 2`。
 3. 找排序逻辑：
 - 从高到低：需要 `-r` (降顺)。
 - 成绩是数字：为了防止 `100` 排在 `9` 后面，必须有 `-n` (数值)。
 4. 结论：选 B。
-

🌟 考试避坑小贴士

在考试中，`sort` 经常和 `uniq` 连用。记住：「`uniq` は直前の行しか比較しない」（`uniq` 只比较相邻行）。所以，如果你想把整个文件去重，一定要先 `sort` 再 `uniq`！

好的，我们来攻克 `uniq` (ユニーク) 命令。结合你上传的图片和 Linux 认证考试的重点，这个命令其实是文本处理中的“查重小能手”。

💡 知识点总结: `uniq` 命令是处理“重复行”的过滤器

`uniq` 的核心功能是「重複した行を 1 行にまとめる」（把重复的行合并为一行）。但请务必记住它的致命弱点：它只能处理相邻的重复行。

1. 核心术语表 (用語集)

根据你上传的图片，这两个选项是考试中区分

`uniq` 行为的关键：

オプション	説明
-d	重複している行のみを表示
-u	重複していない行のみを表示

日語术语	中文含义	记忆关键点
重複している行のみを表示	仅显示重复过的行	使用 -d (Duplicated)
重複していない行のみを表示	仅显示没有重复过的行	使用 -u (Unique)
隣接した行 (りんせつ)	相邻的行	<code>uniq</code> 处理的前提条件
連続した重複	连续的重复	只有连续出现， <code>uniq</code> 才会发现它

2. 🏠 小学生记忆法：“火眼金睛的检查员”

想象 `uniq` 是一个站在传送带旁边的检查员，产品一个接一个从他面前经过。

- 它的性格：记性特别短！它只记得“刚刚过去的那一个”是什么。
 - 如果现在过来一个 `Black`，刚才也是 `Black`，它就会说：“这个我看过了，扔掉！”
 - 如果中间隔了一个 `White`，它就会忘记之前的 `Black`，导致重复行无法被合并。所以，必须先排好队 (`sort`)，让一模一样的东西挨在一起。
- `-d` (Duplicated) —— “抓现行”
 - 检查员只对那些“成双成对”出现的捣蛋鬼感兴趣。如果一个产品是孤零零的，他直接无视。只有看到重复出现的東西，他才会把它抓出来。
 - 记法：D = Duplicate (双胞胎)。
- `-u` (Unique) —— “找独苗”

- 检查员这次的任务是寻找“世界上唯一的花”。只要某个产品出现了两次或以上，他就觉得“这不唯一了”，统统扔掉。最后只留下那些**从来没重复过的**。
- **记法：U = Unique** (唯一的)。

3. ✂️ 实战演习 (题目模拟)

问题： 现有文件内容如下，执行 `uniq -d` 的结果是什么？

Plaintext

代码块

```
1  Apple
2  Orange
3  Orange
4  Banana
```

- A. Apple , Orange , Banana (各显示一次)
- B. Orange
- C. Apple , Banana

解析：

1. **关键词：** `-d` 代表「重複している行のみ」。
2. Apple 只出现一次，不显示。
3. Orange 连续出现了两次，符合“重复”条件，显示。
4. Banana 只出现一次，不显示。
5. **结论：** 选 B。

4. 考试避坑指南

- **先 `sort` 后 `uniq`：** 考试如果问“如何统计文件中所有不重复的行”，选项里只有 `uniq` 是不够的，必须是 `sort File | uniq`。
- **`-c` 选项 (补充)：** 图片里没写，但考试常考。 `-c` (Count) 会在每行前面显示「出現回数」(出现的次数)。

💡 知识点总结： `cut` 命令是精准切分文本的“手术刀”

`cut` 的核心功能是「テキストの各行から一部分を取り出す」（从每一行中提取一部分）。它最常用于处理像 `/etc/passwd`（冒号分隔）或 CSV（逗号分隔）这样的结构化文件。

1. 核心术语表 (用語集)

根据你提供的图片（最后一张），这是 `cut` 的三大必考参数：

オプション	説明
-c 文字数	抽出する文字位置を指定
-d 区切り文字	区切り文字を指定(デフォルトはタブ)
-f フィールド	抽出するフィールドを指定 n → n番目のフィールド n,m → n番目とm番目のフィールド

日语术语	中文含义	考试关键点
区切り文字 (くぎりもじ)	分隔符	使用 -d (Delimiter) 指定，默认为 Tab
抽出するフィールド	提取的字段/列	使用 -f (Field) 指定第几列
文字位置を指定	指定字符位置	使用 -c (Character) 按字符个数切

2. 🏠 小学生记忆法：“切香肠的小厨师”

想象 `cut` 是一个在厨房里切香肠的小厨师。

- d (Delimiter) —— “切在哪里？”**
 - 香肠上有很多节。 `-d` 就像是厨师在找那个「縫隙」。
 - 如果是冒号分隔的文件，就告诉厨师： `-d ':'` （看到冒号就切一刀）。
 - 记法： **D = Divide** (分开) 或者 **Delimiter**。
- f (Field) —— “要哪一段？”**
 - 香肠切成一段一段后，你想要第几段？
 - `-f 1` 是要第一段， `-f 1,3` 是要第一和第三段。
 - 记法： **F = Fruit** (果实/我们要的东西)。
- c (Character) —— “不用縫隙，直接量长度”**
 - 如果不看縫隙，直接拿尺子量，每段切 5 厘米。这就是 `-c 1-5`。
 - 记法： **C = Cut** (直接剪下第几个字)。

3. 🗂️ 实战演习 (结合图片逻辑)

场景：假设文件 `test.txt` 内容是 `root:x:0:0`。

- 问题：如何提取第一列（用户名）？
 - 解题步骤：
 - a. 确认分隔符是冒号：`-d ':'`
 - b. 确认要的是第一列：`-f 1`
 - c. 组合命令：`cut -d ':' -f 1 test.txt` → 结果得到 `root`。
-

4. 🧠 综合记忆：三剑客的配合

既然你上传了所有图片的汇总，我们用一句话把它们串起来：

1. **sort** (排队官)：先把乱糟糟的行排好队 (比如 `-n` 按数字排)。
2. **uniq** (查重员)：把排好队后挨在一起的重复行处理掉 (比如 `-d` 抓双胞胎)。
3. **cut** (手术刀)：最后把剩下的唯一行切开，只拿走你关心的那一列 (`-f`)。

考试高频组合拳：

```
cat file | sort | uniq | cut -f 1
```

(排序 → 去重 → 拿走第一列)

💡 知识点总结：数据搬运的“箭头指引”

重定向就是改变数据的“出口”和“入口”。默认情况下，命令从键盘读入，结果显示在屏幕。重定向符号就像导流管，把这些数据导向文件或者“黑洞” (`/dev/null`)。

記号	説明
<	標準入力の入力元を指定 (0< と同義)
<<	標準入力の入力元を指定し、終了文字まで入力 (0<< と同義)
>	標準出力の出力先を指定 (1> と同義)
>>	標準出力の出力先を指定し、出力先に追記 (1>> と同義)
2>	標準エラー出力の出力先を指定
2>>	標準エラー出力の出力先を指定し、出力先に追記
>&2	標準出力の出力先を、標準エラー出力の出力先と同じにする (1>&2 と同義)
2>&1	標準エラー出力の出力先を、標準出力の出力先と同じにする

小学生记忆法：给数据“搬家”

我们可以把这些符号看作不同的搬家小车：

输入组：把东西搬进屋子

- < (或者 0<)：“吸尘器”。把文件里的东西吸出来给命令吃。
- <<：“到此为止”。命令一直吃，直到看到你设置的那个“结束暗号”才停下。

输出组：把结果搬出屋子

- > (或者 1>)：“霸道总裁”。搬新家具进屋前，先把旧的一脚踢开（覆盖）。
- >>：“排队小朋友”。很有礼貌，把新家具放在旧家具后面（追加）。

报错组：专门运送“坏消息”

- 2>：“故障车”。只负责搬运报错的信息。
- 2>>：“故障登记本”。把新的错误信息写在旧的错误后面。

合体组：把两辆车并在一起

- 2>&1：“影子跟随”。让“故障车” (2) 跟着“正常车” (1) 走同一个出口。
- >&2：“反向跟随”。让“正常话”跟着“报错声”一起走。

5. 实战演习：考试怎么考？

场景 1：你想把一个命令的所有结果（包括报错）都存进一个文件里。

- **错误写法：** `ls /no_dir > all.txt` （报错信息还是会跳到屏幕上）。
- **正确写法：** `ls /no_dir > all.txt 2>&1` （先让正常结果去文件，再让错误信息跟着正常结果去文件）。

场景 2：你想运行命令，但不想看任何结果，也不想看报错。

- **必考命令：** `command > /dev/null 2>&1`。
- **解释：** `/dev/null` 是 Linux 的“黑洞”，丢进去就消失了。

☀ 考试避坑指南

1. **数字的含义：**看到 `1` 联想“正确”，看到 `2` 联想“错误”。
2. **箭头的数量：**一个箭头 `>` 会清空文件，两个箭头 `>>` 才是安全的添加。
3. **注意 `2>&1` 的顺序：**通常要把 `> file` 写在前面，再写 `2>&1`。

这是一道关于 **スワップ領域 (交换空间)** 的核心概念题，在 Linux 认证考试中，它属于「**ストレージ管理 (存储管理)**」的高频考点。

💡 知识点总结：スワップ (Swap) 是内存的“替补队员”

当物理内存（RAM）不够用时，Linux 会把一部分硬盘空间当作内存来使，这个区域就叫 **スワップ領域**。它不是普通的文件系统，所以管理方式与普通磁盘分区完全不同。

6. 核心术语表 (用語集)

日语术语	中文含义	考试关键逻辑
スワップ領域	交换空间	硬盘上的虚拟内存区域
有効化 (ゆうこうか)	启用/激活	让 Swap 开始工作
マウント (Mount)	挂载	Swap 不需要 挂载到某个目录
起動時 (きどうじ)	启动时	系统开机自动加载的配置
専用パーティション	专用分区	Swap 通常有自己的分区格式

2. 🏠 小学生记忆法：“书桌与小仓库”

想象你在写作业：

- **物理内存 (RAM)：**就是你的「**书桌**」。书桌越大，你能同时摆开的书越多，干活越快。

- スワップ (Swap): 就是你脚下的「小仓库」。

重点来了:

1. 它不是桌子的一部分: 你不能把仓库“挂”在桌角上 (不用 mount), 它是一个独立的空间。
2. 怎么让它生效?: 如果你在「清单」 (/etc/fstab) 里写好了“我有这个仓库”, 那么你每天早上起床开始写作业时 (启动时), 这个仓库就会自动准备好给你用。
3. 怎么检查它?: 看书桌空间用 df (Disk Free), 但看仓库 (Swap) 得用专门的工具 free 或者 swapon -s。

3. ✂️ 实战演习 (题目解析)

问题: 关于 スワップ領域, 正确的说明是哪一个?

- ❌ A. dfコマンドで使用状況を確認できる
 - 解析: df 是用来查看“带目录挂载的文件系统”的。Swap 没有目录, 所以 df 看不到它。查看 Swap 要用 free 或 swapon -s。
- ❌ B. swapoffコマンドで有効化する
 - 解析: 名字暴露了它! off 是关闭。swapon 才是 有効化 (启用), swapoff 是禁用。
- ❌ C. mount -t swapでマウントする
 - 解析: Swap 「マウント不要」 (不需要挂载)。它是直接交给系统内核管理的, 没有挂载点 (如 /mnt)。
- ✅ D. /etc/fstabに記載があれば起動時に有効化される
 - 解析: 这是标准做法。只要在 /etc/fstab 配置文件里写了 Swap 分区的信息, 系统 起動時 就会自动执行 swapon -a 来激活它。

4. 考试避坑指南 (考试直感)

- 看到 mount 选 Swap → 必错。
- 看到 df 查 Swap → 必错。
- 启用命令: 记住是 swapon。
- 建立命令: 如果要从零建立 Swap, 顺序是: mkswap (格式化) → swapon (启用)。

这道题目考查的是如何结合「正規表現 (正则表达式)」与「テキスト処理コマンド (文本处理命令)」来筛选并统计特定格式的行。

这一知识点主要考察在 Linux 环境下，如何利用 `grep` 匹配以指定字符开头的行，并使用 `wc` 或 `grep` 自带的功能进行数量统计。

💡 知识点总结

在 Linux 中，统计匹配行数主要有两种标准做法：

- 1. **管道组合法**：先用 `grep` 过滤出目标行，再通过 **パイプ (|)** 传给 `wc -l` (Word Count - Lines) 进行计数。
- 2. **原生参数法**：直接使用 `grep -c` (count) 参数，它会直接输出匹配到的行数，效率更高。

🔍 选项解析与详细说明

- **✅ `grep ^# File | wc -l`：正确答案 1。**
 - **理由**：`^#` 是正则表达式，表示匹配 **行頭 (行首)** 是 `#` 的行。通过管道传给 `wc -l`，`wc` 会计算并输出这些行的总数。
- **❌ `grep #$ File | ls -l`：错误。**
 - **理由**：`#$` 匹配的是以 `#` **行末 (结尾)** 的行。此外，`ls -l` 是查看文件列表属性的命令，完全不能用来统计文本行数。
- **❌ `find ^# File | ls -l`：错误。**
 - **理由**：`find` 是用来 **ファイル検索 (搜索文件)** 的命令，不是处理文件内容的工具。且 `ls -l` 的用法同样错误。
- **✅ `grep -c ^# File`：正确答案 2。**
 - **理由**：这是最简洁的做法。`-c` 参数代表「**カウント (Count)**」，它会自动统计并返回匹配 `^#` 的行数，不需要配合 `wc`。
- **❌ `find #$ File | wc -l`：错误。**
 - **理由**：同上，`find` 无法读取文件内部内容，这里的逻辑完全不通。

📊 统计匹配行数的两种方案对比

方案	命令示例	特点
组合式	<code>`grep '模式' 文件`</code>	<code>wc -l`</code>
内置式	<code>grep -c '模式' 文件</code>	简洁、高效 ，在脚本和考试中非常常见。

🧠 小学生级记忆法：找“井号”开头的人

想象你在给一个班的小朋友点名，任务是数出多少人穿着带“#”标志的衣服。

1. 两步走 (`grep | wc`):
2. 你先喊：“穿‘#’衣服的小朋友出列站一排！” (`grep ^#`)。
3. 然后你再走过去，一个一个数这排有多少人 (`wc -l`)。
4. 一步到位 (`grep -c`):
5. 你手里拿着一个计数器 (`-c`)，你一边喊：“穿‘#’衣服的人举手！”一边直接按计数器得出总数。

注意：必须是 `^#` (帽子戴在井号头上)，这代表井号在最前面。如果是 `#$`，那是井号掉进裤兜里了 (在行尾)。

✅ 正确答案

1. `grep ^# File | wc -l`
2. `grep -c ^# File`

📝 考试小秘籍

- **正则符号的定位：**在 LPI 考试中，`^` (行头) 和 `$` (行末) 是必考的 **メタキャラクタ (元字符)**。
- **多选陷阱：**当你看到选项中有 `wc -l` 和 `grep -c` 同时出现且正则部分一致时，它们往往就是那两个正确答案。
- **区分命令职能：**`find` 找文件，`grep` 找内容，`wc` 搞统计。千万别混用！

这一知识点主要考察在 Linux 环境下，如何利用 `grep` 命令的 **オプション (选项)** 来精准过滤和统计文本内容。结合你提供的表格，这是 LPI 考试中出现频率最高的基础技能。

💡 知识点总结

`grep` 的强大之处在于它的 **オプション (选项)**。通过不同的参数，你可以决定是只看行数、忽略大小写，还是反向筛选。在处理日志文件或统计数据时，灵活组合这些参数是提高效率的关键。

🔍 选项解析与详细说明 `grep` コマンド

`grep` [オプション] 検索パターン [ファイル名]

根据你上传的 `grep` 选项表，以下是考试中最核心的几个功能：

- `-c` (Count):
 - 理由：「マッチした行の行数のみ表示」（仅显示匹配行的行数）。它是统计任务的首选，可以代替 `wc -l` 减少一次管道操作。
- `-i` (Ignore case):
 - 理由：「大文字と小文字を区別しない」（不区分大小写）。在搜索 "Error" 时，它能同时帮你找出 "error" 和 "ERROR"。
- `-v` (Invert match):
 - 理由：「マッチしなかった行を表示」（显示不匹配的行）。常用于排除干扰项，比如排除所有注释行。
- `-n` (Line number):
 - 理由：「先頭に行番号をつけて表示」（在前面带上行号显示）。帮你快速定位匹配内容在文件中的具体位置。
- `-E` (Extended regex):
 - 理由：「拡張正規表現を使用」（使用扩展正则表达式）。支持更复杂的匹配逻辑（如 `|` 代表“或”）。

オプション	説明
-c	マッチした行の行数のみ表示
-f	検索パターンをファイルから読み込む
-i	大文字と小文字を区別しない
-n	先頭に行番号をつけて、マッチした行を表示
-v	マッチしなかった行を表示
-E	拡張正規表現を使用(egrepコマンドと同様)
-F	検索パターンを正規表現ではなく、固定文字列とする(fgrepコマンドと同様)

 `grep` 常用功能排查表

需求 (日本語)	对应选项	记忆关键点
行数を知りたい	-c	Count (计数)
大小文字を無視したい	-i	Ignore (忽略)
特定行を除外したい	-v	Void (排除/反转)
場所を特定したい	-n	Number (行号)

 小学生级记忆法：“超级搜索小侦探”

想象 `grep` 是一个在图书馆找书的小侦探，这些选项就是他的**特殊技能卡**：

1. **-c 计数卡**：侦探不把书拿出来，只是对着书架数一数：“一共有 5 本符合要求！”
2. **-i 色盲卡**：侦探不管书皮是“红色”还是“粉红色”，只要是“红系”的都算（不分大小写）。
3. **-v 讨厌鬼卡**：侦探讨厌某种书。你告诉他“讨厌恐怖书”，他就会把除了恐怖书以外的所有书都给你。
4. **-n 门牌号卡**：侦探每找出一本书，都会大喊一声：“这本书在第 42 号书架！”（带行号显示）。

✅ 正确答案 (针对前文题目)

在上一题中，我们需要统计行数，最标准的用法就是：

```
grep -c ^# File
```

📖 考试小秘籍

- **字母含义关联**：记住 `-v` 代表 **InVert** (反向)，`-i` 代表 **Ignore** (忽略)，这样在考试时即使紧张也能靠字母猜出大概。
- **组合使用**：选项可以叠buff，比如 `grep -iv` 代表“忽略大小写且反向排除”。
- **正则配合**：选项决定“怎么搜”，而后面的 **正規表現** (如 `^` 或 `$`) 决定“搜什么”。

这道题目考察的是在 Linux 中查看磁盘分区 **UUID (Universally Unique Identifier)** 的常用工具。

这一知识点主要考察如何获取分区的唯一识别码 (UUID)，这在配置 `/etc/fstab` 自动挂载时非常关键。

💡 知识点总结

在 Linux 系统管理中，UUID 是分区的身份证。为了获取这个信息，通常有两条路径：

1. **专用查询工具**：直接读取块设备属性。
2. **列表显示工具**：在列出磁盘分区信息时，通过参数显示其属性。

🔍 选项解析与详细说明

- **✅ `lsblk`：正确答案 1。**

- 理由: `lsblk` (list block devices) 用于列出所有可用块设备的信息。虽然默认输出不显示 UUID, 但通过使用 `-f` (file system) 选项, 可以清晰地看到每个分区的 **UUID**、文件系统类型和挂载点。
- ✗ `showblk`: 错误。
 - 理由: Linux 系统中不存在名为 `showblk` 的标准命令。这通常是考试中常见的“干扰项”, 看起来很专业但实际不存在。
- ✗ `mount`: 错误。
 - 理由: `mount` 命令主要用于显示当前**已挂载**的文件系统及其参数。虽然它能显示设备名和挂载点, 但通常不会直接输出设备的 UUID。
- ✗ `ps`: 错误。
 - 理由: `ps` (process status) 是用于查看当前系统中运行的 **プロセス (进程)** 状态的命令, 与磁盘设备信息无关。
- ✓ `blkid`: 正确答案 2。
 - 理由: `blkid` (block id) 是最直接的专用命令。它专门用于查询块设备的属性, 执行后会直接列出所有分区的名称、**UUID** 以及文件系统类型 (TYPE) 。

 UUID 查询工具对比表

命令	常用参数	作用 (日本語)	优点
<code>blkid</code>	(无)	デバイスの属性を表示	最直接 , 输出简洁明了
<code>lsblk</code>	<code>-f</code>	デバイスをツリー状に表示	可视化强 , 能看到磁盘的树状结构

 小学生级记忆法：“找身份证”

想象每个磁盘分区都是一个小朋友, UUID 就是他们的**身份证号**:

- `blkid` (Block ID):
- 就像警察叔叔手里拿的**身份证扫描仪**。扫描仪对准谁, 谁的身份证 ID 就会直接显示在屏幕上。
- `lsblk -f` (List Block):
- 就像班主任手里的**学生花名册**。原本花名册只写了名字, 但只要班主任翻到“详细页” (`-f`), 就能在名字旁边看到每个人的身份证号。

✅ 正确答案

1. lsblk
2. blkid

📝 考试小秘籍

- **关键参数：**记住 `lsblk` 必须配合 `-f` 才能在实操中看到 UUID，但在多选题中，看到它作为查询工具通常是正确的。
- **辨别虚假命令：**Linux 很多查询命令以 `ls` (list) 开头。如果看到没见过的 `show...` 开头的命令，大概率是诱导选项。
- **root 权限：**在实际环境中，`blkid` 通常需要 root 权限才能看到所有信息，考试时若题目涉及权限问题需留意。

这道题目考察的是使用 `sed` 命令配合 **正規表現 (正则表达式)** 来删除特定行（空行）的操作。

这一知识点主要考察如何识别「**空行 (空白行)**」。在正则表达式中，空行被定义为“行首和行末之间没有任何字符”的状态。

💡 知识点总结

`sed` 命令的 `/pattern/d` 格式用于删除匹配 `pattern` 的行。要删除空行，我们需要一个能代表“空”的正则表达式。

🔍 选项解析与详细说明

- **✅ `^$`：正确答案。**
 - **理由：**`^` 表示 **行頭 (行首)**，`$` 表示 **行末 (行末)**。当这两个符号连在一起时，意味着从行开始到行结束之间没有任何内容，即 **空行**。
- **❌ `\n`：错误。**
 - **理由：**`\n` 代表 **改行文字 (换行符)**。虽然空行包含换行符，但在 `sed` 的模式空间中，通常不直接使用 `\n` 来匹配整行内容。
- **❌ `&$`：错误。**
 - **理由：**在 `sed` 的替换部分，`&` 代表匹配到的字符串，但在匹配模式中，它没有特殊含义。这是一个干扰项。
- **❌ `^`：错误。**
 - **理由：**这只代表 **行頭**。如果只写这个，`sed` 会尝试删除所有有开头的行（即所有行）。

- ✖ `^#`：错误。
 - 理由：这匹配的是 `コメント行` (注释行)，即以 `#` 开头的行。

 常用正则元字符 (メタキャラクタ) 对比表

符号 (日本語)	中文含义	记忆逻辑
<code>^</code> (行頭)	行首	像个尖尖的帽子戴在头顶。
<code>\$</code> (行末)	行末	像美元符号作为工资发在最后。
<code>^\$</code> (空行)	空行	头和尾贴在一起，中间啥也没有。
<code>^#</code> (コメント)	注释行	井号在最前面。

 小学生级记忆法：“亲嘴的符号”

想象每一行文字都有一个“头” (`^`) 和一个“尾” (`$`)。

- 正常行：头和尾之间隔着很多字。
- 空行：这一行一个字都没有，所以“头” (`^`) 和“尾” (`$`) 就会直接碰到一起，像在“亲嘴”一样。

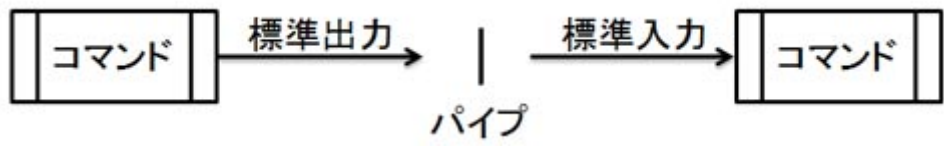
所以，当你看到两个符号紧紧贴在一起 `^$`，那就是在找那个空空如也的房间（空行）！

 正确答案

`^$`

 考试小秘籍

- `sed` 的 `d` 命令：记住 `d` 就是 **Delete** (削除)。
- 排除空格行：注意 `^$` 只能删除完全没有内容的行。如果一行里有空格，它就不算空行了。如果想连带空格行一起删，考试中可能会考 `^[[:space:]]*$`。
- `grep` 也可以用：统计空行也可以用 `grep -c '^$' test.txt`。



这一知识点主要考察如何使用 `wc` (Word Count) 命令及其特定的 **オプション** (选项) 来统计文本文件的行数。

💡 知识点总结

`wc` 命令用于统计文件的 **行数 (Lines)**、**单词数 (Words)** 和 **字节数 (Bytes)**。如果不带参数执行 `wc`，它会同时输出这三项数据；如果带了选项（如 `-l`），则只输出请求的那一项数据。

🔍 选项解析与详细说明

- ✅ 「file.txt」ファイルの行数：正确答案。
 - 理由：`wc` 命令的 `-l` 选项代表 **Lines (行)**。通过管道 (`|`) 接收 `cat` 的内容后，它会统计总共有多少个换行符，从而得出文件的总行数。
- ❌ エラーになる：错误。
 - 理由：虽然通常直接使用 `wc -l file.txt` 更高效，但 `cat file.txt | wc -l` 这种管道组合在 Linux 中是合法且极其常见的操作方式。
- ❌ 「file.txt」ファイルの文字(バイト)数：错误。
 - 理由：统计字节数需要使用 `-c` (Bytes/Characters) 选项。
- ❌ 「file.txt」ファイルの行数・単語数・文字(バイト)数：错误。
 - 理由：只有在 **不添加任何选项** 执行 `wc` 时，才会同时显示这三项。
- ❌ 「file.txt」ファイルの単語数：错误。
 - 理由：统计单词数需要使用 `-w` (Words) 选项。

📊 `wc` 常用选项对比表

选项 (日本語)	作用 (中文)	英文原意	记忆点
-l	行数	Line	List 的行
-w	単語数	Word	What 单词
-c	バイト数	Character/Byte	Count 字节

🧠 小学生级记忆法：“数数小能手”

想象 `wc` 是一个坐在教室门口数数的小能手：

1. `-l` (Line)：他盯着「门槛」。每过去一个人（一行），他就按一下计数器。最后告诉你一共过去了多少行。
2. `-w` (Word)：他盯着「书包」。数一数每个人背了多少个单词包。
3. `-c` (Character)：他盯着「纽扣」。把每个人衣服上所有的扣子（字节）都数一遍。

由于题目里用了 `-l`，所以小能手只数了「行数」。

✅ 正确答案

「file.txt」ファイルの行数

📝 考试小秘籍

- 管道 vs 直接读取：
 - `cat file.txt | wc -l` → 只输出 **数字**（因为文件名信息在管道里丢失了）。
 - `wc -l file.txt` → 输出 **数字 + 文件名**。
- 多选预防：考试有时会考“统计字符数”或“单词数”，记住 `l` (Line)、`w` (Word)、`c` (Character) 这三个首字母即可稳拿分数。

这道题目考察的是在没有预设配置文件（`/etc/fstab`）的情况下，如何手动执行 **マウント (Mount)** 操作。

这一知识点主要考察 `mount` 命令的基础语法，即如何将一个 **デバイス (设备)** 关联到文件系统中的 **マウントポイント (挂载点)** 目录上。

💡 知识点总结

`mount` 命令的作用是让存储设备（如 CD-ROM、USB、硬盘分区）的内容在 Linux 的目录树中变得可见。

- 基本语法：`mount [选项] <设备名> <挂载点目录>`
- 如果 `/etc/fstab` 中没有记录，你必须同时告诉系统“谁 (Device)”和“哪 (Directory)”。

🔍 选项解析与详细说明

- ✗ `mount /mnt/cdrom :`
 - 理由: 只有在 `/etc/fstab` 中预先写好了 `/mnt/cdrom` 对应的设备时, 才能只写挂载点。题目明确说了没有添加エントリ。
- ✗ `mount /dev/sr0 :`
 - 理由: 同上。如果没有配置记录, 系统不知道要把这个设备挂载到哪个目录下。
- ✓ `mount /dev/sr0 /mnt/cdrom : 正确答案。`
 - 理由: 这是标准的完整语法。第一个参数是源设备 `/dev/sr0` , 第二个参数是目标目录 `/mnt/cdrom` 。
- ✗ `mount -t ext4 /dev/sr0 /mnt/cdrom :`
 - 理由: 虽然语法格式正确, 但 `ext4` 是硬盘常用的文件系统。CD-ROM 通常使用 `iso9660` 或 `udf`。强制指定错误的类型会导致报错。通常不加 `-t` 让系统自动识别更安全。
- ✗ `mount /mnt/cdrom /dev/sr0 :`
 - 理由: 顺序反了! 必须是 “设备在前, 目录在后” 。

 手动挂载 (Manual Mount) 的要素

要素	示例	角色 (日本語)	说明
Command	mount	命令本体	执行挂载动作
Device	/dev/sr0	マウント元	实际的硬件设备或分区
Directory	/mnt/cdrom	マウント先	访问设备内容的入口 (必须已存在)

 小学生级记忆法: “插线板与插头”

想象你有一个 插头 (CD-ROM 设备) 和一个 插座 (/mnt/cdrom 目录) :

- 挂载动作: 就是把插头插进插座里。
- 命令顺序: 你必须拿着 插头 (`/dev/sr0`) 去找 插座 (`/mnt/cdrom`)。
- 如果没配置 (`fstab`): 你手里必须同时有这两样东西, 缺一不可。如果你只拿着插头 (设备) 或者只看着插座 (目录) , 电是怎么也通不了的。

✓ 正确答案

考试小秘籍

- **顺序原则：**记住“**从左往右，从小到大**”。物理设备（小）挂载到目录树（大）中。
- **自动识别：**现代 Linux 的 `mount` 命令非常聪明，通常不需要显式使用 `-t` 指定文件系统类型。
- **卸载命令：**对应的卸载命令是 `umount`（注意：**不是** `unmount`，没有第一个 n）。你可以执行 `umount /dev/sr0` 或 `umount /mnt/cdrom`，两者择其一即可。

这道题目考察的是在没有预设配置文件（`/etc/fstab`）的情况下，如何手动执行 **マウント (Mount)** 操作。

这一知识点主要考察 `mount` 命令的基础语法，即如何将一个 **デバイス (设备)** 关联到文件系统中的 **マウントポイント (挂载点)** 目录上。

知识点总结

`mount` 命令的作用是让存储设备（如 CD-ROM、USB、硬盘分区）的内容在 Linux 的目录树中变得可见。

- **基本语法：** `mount [选项] <设备名> <挂载点目录>`
- 如果 `/etc/fstab` 中没有记录，你必须同时告诉系统“**谁 (Device)**”和“**哪 (Directory)**”。

选项解析与详细说明

-  `mount /mnt/cdrom :`
 - **理由：**只有在 `/etc/fstab` 中预先写好了 `/mnt/cdrom` 对应的设备时，才能只写挂载点。题目明确说了没有添加エントリ。
-  `mount /dev/sr0 :`
 - **理由：**同上。如果没有配置记录，系统不知道要把这个设备挂载到哪个目录下。
-  `mount /dev/sr0 /mnt/cdrom`：**正确答案。**
 - **理由：**这是标准的完整语法。第一个参数是源设备 `/dev/sr0`，第二个参数是目标目录 `/mnt/cdrom`。
-  `mount -t ext4 /dev/sr0 /mnt/cdrom :`
 - **理由：**虽然语法格式正确，但 `ext4` 是硬盘常用的文件系统。CD-ROM 通常使用 `iso9660` 或 `udf`。强制指定错误的类型会导致报错。通常不加 `-t` 让系统自动识别更安全。
-  `mount /mnt/cdrom /dev/sr0 :`
 - **理由：**顺序反了！必须是“**设备在前，目录在后**”。

手动挂载 (Manual Mount) 的要素

要素	示例	角色 (日本語)	说明
Command	mount	命令本体	执行挂载动作
Device	/dev/sr0	マウント元	实际的硬件设备或分区
Directory	/mnt/cdrom	マウント先	访问设备内容的入口（必须已存在）

小学生级记忆法：“插线板与插头”

想象你有一个 **插头**（CD-ROM 设备） 和一个 **插座**（/mnt/cdrom 目录）：

1. **挂载动作**：就是把插头插进插座里。
2. **命令顺序**：你必须拿着 **插头**（ /dev/sr0 ）去找 **插座**（ /mnt/cdrom ）。
3. **如果没配置（fstab）**：你手里必须同时有这两样东西，缺一不可。如果你只拿着插头（设备）或者只看着插座（目录），电是怎么也通不了的。

正确答案

`mount /dev/sr0 /mnt/cdrom`

考试小秘籍

- **顺序原则**：记住 “**从左往右，从小到大**”。物理设备（小）挂载到目录树（大）中。
- **自动识别**：现代 Linux 的 `mount` 命令非常聪明，通常不需要显式使用 `-t` 指定文件系统类型。
- **卸载命令**：对应的卸载命令是 `umount`（注意：**不是** `unmount`，没有第一个 n）。你可以执行 `umount /dev/sr0` 或 `umount /mnt/cdrom`，两者择其一即可。

这一知识点主要考察在 Linux 环境下，如何通过 `mount` 命令的 **オプション（选项）** 来控制文件系统的挂载行为。掌握这些选项是管理磁盘分区和 `/etc/fstab` 配置文件的基础。

知识点总结

`mount` 命令用于将设备（如硬盘、CD-ROM）连接到 Linux 的目录树中。图片中展示的三个选项是考试中出现频率最高的：

- `-a`：一键挂载。根据配置文件自动操作。
- `-o`：细节控制。指定读写权限等特殊属性。
- `-t`：指明种类。明确告诉系统文件系统的类型（如 ext4, xfs, iso9660）。

🔍 选项解析与详细说明

根据你上传的 `mount` 选项表（最后两张图内容一致）：

- ☒ `-a` (all):
 - 理由：「`/etc/fstab`」ファイルに記載されているファイルシステムをすべてマウント（挂载 `/etc/fstab` 文件中记录的所有文件系统）。
 - 注意点：如果配置中写了 `noauto` 选项，则会被排除在自动挂载之外。
- ☒ `-o` (options):
 - 理由：続けてマウントオプションを指定（随后指定具体的挂载选项）。
 - 常见组合：例如 `-o ro`（只读）或 `-o rw`（读写）。它是定制化挂载的“开关盒”。
- ☒ `-t` (type):
 - 理由：続けてファイルシステムの種類を指定（随后指定文件系统的种类）。
 - 补充：如果不使用此选项，Linux 会尝试自动推测（自動で種類を推測）。但在挂载老旧设备或特定格式（如 CD-ROM 的 `iso9660`）时，手动指定更安全。

📊 `mount` 常用选项对比表

选项	中文功能	典型用法	记忆点
<code>-a</code>	全部挂载	<code>mount -a</code>	All (全部)
<code>-o</code>	特定选项	<code>mount -o ro /dev/sdb1 /mnt</code>	Options (选项)
<code>-t</code>	指定类型	<code>mount -t ext4 /dev/sdb1 /mnt</code>	Type (类型)

オプション	説明
<code>-a</code>	「 <code>/etc/fstab</code> 」ファイルに記載されているファイルシステムをすべてマウント（ <code>noauto</code> マウントオプションが指定されているものは除く）
<code>-o</code>	続けてマウントオプションを指定
<code>-t</code>	続けてファイルシステムの種類を指定（指定しない場合は自動で種類を推測）

🧠 小学生级记忆法：“插线板的三种模式”

想象 `mount` 命令就像是在给家里的电器（设备）插上“插线板”（目录）：

1. **-a (All-in-one): “总闸开关”**。你按一下总闸，所有在清单（`/etc/fstab`）里的电器全都通电工作。
 2. **-o (Option): “调速器”**。你不仅插上了电，还拨动了一个开关，说：“这个风扇只能转，不能倒转（只读模式 `ro`）”。
 3. **-t (Type): “转换插头”**。有的电器是圆头的，有的是扁头的。你得明确告诉插线板：“这是 `ext4` 类型的电器，请用对应的电流”。
-

✅ 正确答案

- **-a**: 挂载 `/etc/fstab` 中除 `noauto` 外的所有项。
 - **-o**: 指定读写、权限等额外属性。
 - **-t**: 明确指定文件系统类型（如 `ext4`）。
-

📝 考试小秘籍

- **`/etc/fstab` 是核心**: 很多关于 `mount -a` 的题目都会关联到这个文件的配置是否正确。
- **顺序别弄反**: `mount -t ext4 /dev/sr0 /mnt`。选项永远紧跟在 `mount` 后面。
- **卸载的坑**: 卸载是 `umount`，它没有 `-t` 或 `-a` 的说法（虽然有 `-a` 卸载所有，但很少考），它通常只需要指定挂载点即可。

